ĐẠI HỌC BÁCH KHOA HÀ NỘI

TRƯỜNG ĐIỆN – ĐIỆN TỬ

BÁO CÁO BÀI TẬP LỚN

Học phần Lập trình nâng cao

ROOMHUB

HỆ THỐNG ĐẶT VÀ QUẢN LÝ PHÒNG HỌC, PHÒNG HỌP

Nhóm thực hiện:	Hoàng Văn Tùng	20252389M
	Đỗ Thùy Linh	20252478M
	Nguyễn Đình Phúc	20251012M
	Nguyễn Việt Dũng	20252690M
Giảng viên hướng dẫn:	TS. Nguyễn Thanh Bình	

https://github.com/xwomen1/classroom-booking

HÀ NỘI, THÁNG 10 NĂM 2026

ĐẠI HỌC BÁCH KHOA HÀ NỘI

TRƯỜNG ĐIỆN – ĐIỆN TỬ

TRANG THÔNG TIN BÁO CÁO

Học phần Lập trình nâng cao

ROOMHUB

HỆ THỐNG ĐẶT VÀ QUẢN LÝ PHÒNG HỌC, PHÒNG HỌP

Nhóm thực hiện:	Hoàng Văn Tùng	20252389M
	Đỗ Thùy Linh	20252478M
	Nguyễn Đình Phúc	20251012M
	Nguyễn Việt Dũng	20252690M
Giảng viên hướng dẫn:	TS. Nguyễn Thanh Bình	

https://github.com/xwomen1/classroom-booking

HÀ NỘI, THÁNG 10 NĂM 2026

LỜI CẢM ƠN

Nhóm RoomHub xin cảm ơn TS. Nguyễn Thanh Bình và các thầy cô đã hỗ trợ nhóm trong quá trình học tập, phát triển bài tập lớn. Nhóm cảm ơn các thành viên đã cùng xây dựng tài liệu nghiệp vụ, triển khai ứng dụng, kiểm thử và hoàn thiện báo cáo. Những nội dung còn hạn chế sẽ được nhóm tiếp tục đối chiếu với mã nguồn và kiểm chứng trong các giai đoạn tiếp theo.

TÓM TẮT NỘI DUNG BÀI TẬP LỚN

Đề tài xuất phát từ nhu cầu giảng viên tìm và đăng ký phòng học cho hoạt động giảng dạy, họp và công việc chuyên môn. Khi thông tin phòng và lịch sử dụng được quản lý bằng file Excel, việc tìm phòng trống, kiểm tra sức chứa và thiết bị, cũng như đăng ký phòng còn phụ thuộc nhiều vào thao tác thủ công. Nhóm phát triển RoomHub để số hóa quy trình tìm phòng, đặt phòng, phê duyệt, thông báo và cấp quyền truy cập cho giảng viên và cán bộ quản trị. Trong phạm vi pre-MVP, hệ thống được mô hình hóa và thử nghiệm trên một tòa nhà tám tầng có cách tổ chức phòng tương tự tòa C7, Đại học Bách khoa Hà Nội; chưa triển khai trên toàn khuôn viên trường.

RoomHub gồm ứng dụng Android React Native/TypeScript, backend Java 17/Spring Boot cung cấp REST API và cơ sở dữ liệu H2 lưu phía máy chủ qua JPA. Cấu hình hiện tại bật chế độ remote: app trao đổi dữ liệu với backend qua API; chế độ local dùng Async Storage vẫn được giữ riêng. Đây là hai chế độ dữ liệu, không phải đồng bộ nền hai chiều giữa bản local offline và server. Các phép kiểm tra quy tắc được tổ chức ở tầng dịch vụ; thao tác ghi phía máy chủ chạy trong giao dịch có khóa bi quan.

Với phòng gắn khóa số, quản trị viên hoặc người dùng đã được cấp quyền theo phòng có thể tạo mã tạm thời gồm bảy chữ số. Ứng dụng gửi bản tin MQTT qua OneIoT khi có kết nối trong phiên; gửi bản tin chưa chứng minh khóa thật đã áp dụng mã. Backend, các API nghiệp vụ, cơ sở dữ liệu H2 và đường app–server đã có trong mã nguồn; việc xác nhận khả năng đồng bộ trên hạ tầng triển khai liên tục và nhiều thiết bị vẫn cần kiểm thử đầu cuối. Bản Release cũng đã được kiểm tra chạy ở chế độ local.

ABSTRACT

The project starts from instructors' need to find and book classrooms for teaching, meetings, and other academic activities. Room information and schedules are currently managed mainly in Excel files, so checking availability, capacity, equipment, and submitting requests still involves manual work. RoomHub digitises the room search, booking, approval, notification, and access-permission workflow for instructors and administrators. In this pre-MVP scope, the team models and tests the system in an eight-storey building with a room layout similar to Building C7 at Hanoi University of Science and Technology; deployment across the whole campus is outside the current scope.

The scope narrows from shared-room resource management to a specific Android classroom and meeting-room booking system, RoomHub. Its implemented system includes a React Native/TypeScript client, a Java 17/Spring Boot backend exposing REST APIs, and an H2 database accessed through JPA. Remote mode is enabled in the current source: the client reads and writes business data through the backend. Local Async Storage remains available as a separate mode; offline local changes are not automatically reconciled with the server. The configured public tunnel is for testing and does not represent production availability.

For rooms equipped with a smart lock, an administrator or an authorised user may create a seven-digit temporary PIN. The application records the PIN and its validity interval before attempting delivery. Once a OneIoT session is connected, the mobile client publishes the corresponding MQTT message. Broker publication is not treated as proof that the physical lock has applied the PIN. Automated application tests and Java integration tests are present; a release build has also been checked on an Android device in local mode. End-to-end availability of the remote server and multi-device data sharing still depend on a running backend, database and reachable API endpoint.

PHÂN CÔNG NHIỆM VỤ

Bảng dưới đây tổng hợp phân công công việc và đóng góp chính của các thành viên trong quá trình thực hiện đề tài RoomHub.

Thành viên	**Phần việc chính**
Hoàng Văn Tùng	Thiết kế khung ứng dụng và module; triển khai giao diện, nghiệp vụ đặt phòng và quyền theo phòng; tích hợp OneIoT và tạo mã tạm thời; đóng gói bản Android.
Đỗ Thùy Linh	Phát triển dịch vụ Java, phần đồng bộ dữ liệu và một số chức năng giao diện; tham gia thiết kế nhận diện ứng dụng.
Nguyễn Đình Phúc	Phát triển đặt phòng lặp, quản lý trường hợp vắng mặt và kiểm thử liên quan đến đặt phòng.
Nguyễn Việt Dũng	Tổng hợp quy trình nghiệp vụ, xây dựng kịch bản kiểm thử và rà soát các luồng chức năng.

MỤC LỤC

1 GIỚI THIỆU ĐỀ TÀI **13**

1.1 Đặt vấn đề . . . 13

1.2 Mục tiêu và phạm vi đề tài . . . 13

1.3 Định hướng giải pháp . . . 13

1.4 Bố cục báo cáo . . . 14

2 KHẢO SÁT VÀ PHÂN TÍCH YÊU CẦU **15**

2.1 Khảo sát hiện trạng . . . 15

2.1.1 Khảo sát người dùng và quy trình hiện hành . . . 15

2.1.2 Khảo sát các giải pháp tương tự . . . 16

2.2 Tổng quan chức năng . . . 16

2.2.1 Biểu đồ use case tổng quát . . . 16

2.2.2 Biểu đồ use case phân rã “Tìm và đặt phòng” . . . 17

2.2.3 Biểu đồ use case phân rã “Duyệt / từ chối / đổi phòng” . . . 18

2.2.4 Quy trình nghiệp vụ . . . 19

2.3 Đặc tả chức năng . . . 20

2.3.1 Đặc tả use case “Đăng nhập” . . . 22

2.3.2 Đặc tả use case “Tìm và đặt phòng” . . . 22

2.3.3 Đặc tả use case “Duyệt yêu cầu đặt phòng” . . . 23

2.3.4 Đặc tả use case “Đổi phòng thay thế” . . . 23

2.3.5 Đặc tả use case “Tạo mã PIN tạm thời” . . . 24

2.4 Yêu cầu phi chức năng . . . 24

3 NỀN TẢNG LÝ THUYẾT VÀ CÔNG NGHỆ SỬ DỤNG **26**

3.1 Lập trình hướng đối tượng với Java 17 . . . 26

3.2 Spring Boot và cơ chế đảo ngược điều khiển . . . 26

3.3 Jakarta Persistence, Hibernate và cơ sở dữ liệu H2 . . . 27

3.4 Kiến trúc REST và định dạng JSON . . . 27

3.5 React Native và TypeScript . . . 27

3.6 Giao thức MQTT, chuẩn oneM2M và mã hóa AES . . . 28

3.7 Mô hình hóa UML và kiểm thử tự động . . . 28

4 PHÂN TÍCH THIẾT KẾ, TRIỂN KHAI VÀ ĐÁNH GIÁ HỆ THỐNG 30
4.1 Thiết kế kiến trúc . . . 30
4.1.1 Lựa chọn kiến trúc phần mềm . . . 30
4.1.2 Thiết kế tổng quan . . . 31
4.1.3 Thiết kế chi tiết gói . . . 32
4.2 Thiết kế chi tiết . . . 32
4.2.1 Thiết kế giao diện . . . 32
4.2.2 Thiết kế lớp . . . 33
4.2.3 Thiết kế cơ sở dữ liệu . . . 36
4.3 Xây dựng ứng dụng . . . 37
4.3.1 Thư viện và công cụ sử dụng . . . 37
4.3.2 Kết quả đạt được . . . 38
4.3.3 Minh họa các chức năng chính . . . 38
4.4 Kiểm thử . . . 40
4.5 Triển khai . . . 41

5 CÁC GIẢI PHÁP VÀ ĐÓNG GÓP NỔI BẬT 43
5.1 Chống trùng lịch bằng phép giao khoảng thời gian và khóa bi quan . . . 43
5.1.1 Vấn đề . . . 43
5.1.2 Giải pháp . . . 43
5.1.3 Kết quả đạt được . . . 44
5.2 Cấp quyền tạo mã PIN theo phòng và tích hợp khóa thông minh . . . 44
5.2.1 Vấn đề . . . 44
5.2.2 Giải pháp . . . 44
5.2.3 Kết quả đạt được . . . 45
5.3 Kiến trúc module hai chế độ dữ liệu . . . 45
5.3.1 Vấn đề . . . 45
5.3.2 Giải pháp . . . 45
5.3.3 Kết quả đạt được . . . 46

6 KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN 47
6.1 Kết luận . . . 47
6.2 Hướng phát triển . . . 47

TÀI LIỆU THAM KHẢO 48

PHỤ LỤC **49**

A ĐẶC TẢ USE CASE BỔ SUNG **50**

A.1 Đặc tả use case “Hủy đặt phòng” 50

A.2 Đặc tả use case “Hẹn lịch nhận khóa và ủy quyền nhận hộ” 50

A.3 Đặc tả use case “Quản lý lịch sử dụng và bảo trì” 51

A.4 Đặc tả use case “Cấu hình nghiệp vụ” 51

A.5 Đặc tả use case “Quản lý SmartLock” 52

A.6 Đặc tả use case “Quản lý tài khoản và quyền theo phòng” 52

B DANH SÁCH API, CÀI ĐẶT VÀ MINH CHỨNG KIỂM THỬ **53**

B.1 Danh sách REST API của dịch vụ Java 53

B.2 Hướng dẫn cài đặt và chạy thử 53

B.3 Biên bản kiểm tra trên thiết bị 54

DANH MỤC HÌNH VẼ

Hình 2.1 Biểu đồ use case tổng quát của hệ thống RoomHub 17
Hình 2.2 Biểu đồ use case phân rã "Tìm và đặt phòng" 18
Hình 2.3 Biểu đồ use case phân rã "Duyệt / từ chối / đổi phòng" 18
Hình 2.4 Biểu đồ hoạt động của quy trình đặt phòng và cấp quyền tiếp cận . . 19
Hình 2.5 Biểu đồ trạng thái yêu cầu đặt phòng và mã PIN tạm thời 20

Hình 4.1 Kiến trúc triển khai của RoomHub theo mã nguồn hiện tại 30
Hình 4.2 Biểu đồ gói của ứng dụng di động RoomHub 31
Hình 4.3 Màn hình đăng nhập và bảng điều khiển theo vai trò 33
Hình 4.4 Trách nhiệm và quan hệ giữa các nhóm lớp cốt lõi của dịch vụ Java . 34
Hình 4.5 Biểu đồ trình tự ca sử dụng "Tìm và đặt phòng" qua dịch vụ Java . . 35
Hình 4.6 Luồng duyệt yêu cầu và tạo mã tạm thời trong bản hiện tại 36
Hình 4.7 Quan hệ giữa các nhóm dữ liệu lõi của RoomHub 37
Hình 4.8 Màn hình tìm phòng theo sơ đồ tầng và gửi yêu cầu đặt phòng . . . 39
Hình 4.9 Màn hình quản lý phòng và lịch bảo trì của quản trị viên 39
Hình 4.10 Chức năng quản trị và cấu hình nghiệp vụ 40

DANH MỤC BẢNG BIỂU

Bảng 2.1 Các cấu hình phòng trong kịch bản nghiệp vụ 15
Bảng 2.2 So sánh các nhóm giải pháp quản lý đặt phòng 16
Bảng 2.3 Các quy tắc nghiệp vụ đã hiện thực trong RoomHub 21
Bảng 2.4 Đặc tả use case UC01 – Đăng nhập 22
Bảng 2.5 Đặc tả use case UC02 – Tìm và đặt phòng 22
Bảng 2.6 Đặc tả use case UC03 – Duyệt yêu cầu đặt phòng 23
Bảng 2.7 Đặc tả use case UC04 – Đổi phòng thay thế 23
Bảng 2.8 Đặc tả use case UC05 – Tạo mã PIN tạm thời 24

Bảng 3.1 Tổng hợp lựa chọn công nghệ theo yêu cầu 29

Bảng 4.1 Nhiệm vụ các module nghiệp vụ của ứng dụng di động 32
Bảng 4.2 Trách nhiệm chính của lớp dịch vụ nghiệp vụ 34
Bảng 4.3 Thông tin chính của yêu cầu đặt phòng 37
Bảng 4.4 Danh sách thư viện và công cụ sử dụng 38
Bảng 4.5 Các thành phần và minh chứng trong mã nguồn 38
Bảng 4.6 Ca kiểm thử chức năng tạo yêu cầu đặt phòng 40
Bảng 4.7 Ca kiểm thử chức năng duyệt yêu cầu và tạo mã PIN 41
Bảng 4.8 Ca kiểm thử chức năng quản trị và phân quyền 41

Bảng 5.1 Ánh xạ thao tác giữa ứng dụng và dịch vụ Java 46

Bảng A.1 Đặc tả use case UC06 – Hủy đặt phòng 50
Bảng A.2 Đặc tả use case UC07 – Hẹn lịch nhận khóa và ủy quyền nhận hộ . . 50
Bảng A.3 Đặc tả use case UC08 – Quản lý lịch sử dụng và bảo trì 51
Bảng A.4 Đặc tả use case UC09 – Cấu hình nghiệp vụ 51
Bảng A.5 Đặc tả use case UC10 – Quản lý SmartLock 52
Bảng A.6 Đặc tả use case UC11 – Quản lý tài khoản và quyền theo phòng . . 52

Bảng B.1 Các nhóm REST API tiêu biểu của dịch vụ Java 53
Bảng B.2 Phạm vi kiểm tra bản Release trên điện thoại Android 54

DANH MỤC THUẬT NGỮ VÀ TỪ VIẾT TẮT

Thuật ngữ	Ý nghĩa
AES	Chuẩn mã hóa nâng cao (Advanced Encryption Standard)
API	Giao diện lập trình ứng dụng (Application Programming Interface)
APK	Gói cài đặt ứng dụng Android (Android Package Kit)
BR	Quy tắc nghiệp vụ (Business Rule)
CRUD	Tạo, đọc, cập nhật, xóa (Create, Read, Update, Delete)
CSVC	Cơ sở vật chất
CTR	Chế độ mã hóa bộ đếm (Counter mode)
DI	Tiêm phụ thuộc (Dependency Injection)
DTO	Đối tượng truyền dữ liệu (Data Transfer Object)
HTTP	Giao thức truyền siêu văn bản (HyperText Transfer Protocol)
IoC	Đảo ngược điều khiển (Inversion of Control)
IoT	Internet vạn vật (Internet of Things)
JPA	Đặc tả lưu trữ Jakarta (Jakarta Persistence API)
JSON	Định dạng dữ liệu JavaScript Object Notation
JVM	Máy ảo Java (Java Virtual Machine)
MQTT	Giao thức nhắn tin xuất bản – đăng ký cho thiết bị IoT (Message Queuing Telemetry Transport)
MVP	Sản phẩm khả dụng tối thiểu (Minimum Viable Product)
ORM	Ánh xạ đối tượng – quan hệ (Object-Relational Mapping)
PIN	Mã số cá nhân dùng để mở khóa (Personal Identification Number)
QoS	Chất lượng dịch vụ (Quality of Service)
REST	Kiểu kiến trúc chuyển trạng thái đại diện (Representational State Transfer)
TLS	Giao thức bảo mật tầng giao vận (Transport Layer Security)
UML	Ngôn ngữ mô hình hóa thống nhất (Unified Modeling Language)
UUID	Định danh duy nhất toàn cục (Universally Unique Identifier)

CHƯƠNG 1. GIỚI THIỆU ĐỀ TÀI

1.1 Đặt vấn đề

Đề tài xuất phát từ nhu cầu thực tế của giảng viên khi cần tìm và đăng ký phòng học phục vụ giảng dạy, họp hoặc các hoạt động chuyên môn. Hiện nay, thông tin phòng và lịch sử dụng chủ yếu được quản lý bằng file Excel. Vì vậy, việc tìm phòng trống, kiểm tra sức chứa và trang thiết bị, cũng như đăng ký sử dụng vẫn phụ thuộc nhiều vào thao tác thủ công.

Từ nhu cầu đó, nhóm phát triển RoomHub nhằm số hóa quy trình tìm phòng, đặt phòng, phê duyệt, thông báo và cấp quyền truy cập. Hệ thống hướng đến hai nhóm người dùng chính là giảng viên và cán bộ quản trị; hỗ trợ quản lý sức chứa, trang thiết bị, lịch sử dụng và hình thức khóa của từng phòng.

1.2 Mục tiêu và phạm vi đề tài

Trong phạm vi phiên bản pre-MVP, nhóm mô hình hóa và thử nghiệm RoomHub trên một tòa nhà tám tầng có cách tổ chức phòng tương tự tòa C7, Đại học Bách khoa Hà Nội. Đề tài chưa triển khai trên toàn bộ khuôn viên trường. Phạm vi hệ thống tập trung vào ứng dụng Android cho giảng viên và cán bộ quản trị, bao gồm tìm và đặt phòng, phê duyệt yêu cầu, thông báo, quản lý phòng và lịch sử dụng, cùng cấp quyền tiếp cận theo hình thức khóa. Các chức năng backend, API và lưu trữ hỗ trợ cho phiên bản này sẽ được trình bày trong phần giải pháp và kiến trúc hệ thống.

1.3 Định hướng giải pháp

Giải pháp hiện thực dùng ứng dụng Android React Native/TypeScript và dịch vụ backend Java 17/Spring Boot. Backend cung cấp REST/JSON API, dùng Jakarta Persistence để lưu dữ liệu vào H2 ở chế độ tệp trong môi trường thử nghiệm. Cấu hình hiện bật `ENABLE_REMOTE_SYNC`, nên khi server và địa chỉ API truy cập được, ứng dụng đọc/ghi dữ liệu nghiệp vụ qua backend; local Async Storage vẫn là chế độ riêng, không tự hòa giải dữ liệu offline. Endpoint công khai đang cấu hình qua tunnel thử nghiệm, chưa phải hạ tầng production. Các thao tác tạo và duyệt yêu cầu được kiểm tra trong giao dịch, có khóa bi quan trên bản ghi cấu hình và phòng. Chi tiết kiến trúc được trình bày tại Chương 4.

Đối với phòng gắn khóa số, việc duyệt yêu cầu cấp quyền tạo mã cho đúng người dùng và đúng phòng, không chuyển người dùng thành quản trị viên hệ thống. Mã tạm thời gồm bảy chữ số, có khoảng hiệu lực dựa trên lịch đặt phòng. Ứng dụng lưu mã trước và gửi bản tin MQTT trực tiếp qua OneIoT khi người dùng đã kết nối trong phiên. Việc khóa thật nhận và áp dụng mã là bước kiểm chứng riêng, không suy ra chỉ từ kết quả gửi bản tin.

1.4 Bố cục báo cáo

Chương 2 phân tích bối cảnh, kịch bản nghiệp vụ, tác nhân, use case và quy tắc vận hành. Chương 3 giải thích công nghệ và cơ sở lý thuyết được dùng. Chương 4 trình bày thiết kế, kết quả triển khai, giao diện và kiểm thử. Chương 5 phân tích các giải pháp kỹ thuật nổi bật. Chương 6 tổng kết kết quả và hướng phát triển.

CHƯƠNG 2. KHẢO SÁT VÀ PHÂN TÍCH YÊU CẦU

Chương 1 đã giới thiệu bài toán. Chương này đối chiếu tài liệu quy trình nghiệp vụ do nhóm xây dựng với kịch bản sử dụng và các nhóm công cụ quản lý lịch phổ biến, sau đó xác định tác nhân, use case và quy tắc cần triển khai. Các nhận định về nhu cầu được dùng làm giả thiết thiết kế, chưa phải kết quả khảo sát định lượng tại một đơn vị cụ thể.

2.1 Khảo sát hiện trạng

2.1.1 Khảo sát người dùng và quy trình hiện hành

Quy trình nhóm hướng tới cải tiến bắt đầu bằng việc giảng viên ghi nhu cầu đặt phòng trong file Excel. Bảng tính lưu được các đăng ký, nhưng chưa tự phát hiện những dòng bị trùng phòng và thời gian; người phụ trách phải kiểm tra thủ công trước khi xác nhận lịch. Vì vậy người mượn khó biết ngay phòng nào còn trống, còn người quản lý phải dò lịch và điều kiện phòng mỗi khi có yêu cầu mới. Đây là vấn đề nghiệp vụ làm cơ sở cho thiết kế, không phải kết quả khảo sát định lượng hay kết luận về mọi đơn vị.

Từ nhu cầu đó, phạm vi được thu hẹp vào bài toán đặt phòng học/phòng họp: giảng viên tra cứu phòng theo thời gian, sức chứa, trang thiết bị và vị trí; cán bộ quản trị xét duyệt yêu cầu và quản lý lịch. Ở phiên bản pre-MVP, nhóm mô hình hóa và thử nghiệm hệ thống trên tòa nhà tám tầng có cách tổ chức phòng tương tự C7, Đại học Bách khoa Hà Nội. Đây là phạm vi mô hình thử nghiệm, chưa phải triển khai trên toàn trường. Các yêu cầu được cụ thể hóa qua tài liệu quy trình, use case, quy tắc nghiệp vụ và kiểm thử; báo cáo không trình bày chúng như kết quả khảo sát định lượng.

Nguồn nghiệp vụ chính là tài liệu *Quy trình nghiệp vụ RoomHub* do nhóm xây dựng [1]. Tài liệu mô tả các bước từ tra cứu, gửi yêu cầu, phê duyệt, tiếp cận phòng tới hoàn tất. Hai phương thức tiếp cận được xét là nhận khóa cơ hoặc thẻ từ và tạo mã tạm thời cho khóa số quản lý trực tuyến. Bảng 2.1 tóm tắt các trường hợp minh họa trong quy trình.

Bảng 2.1. Các cấu hình phòng trong kịch bản nghiệp vụ

Loại	Phương thức	Quy trình tiếp cận
Loại A	Chỉ dùng mã số	Nhân viên cơ sở vật chất đặt mã tại khóa trước giờ sử dụng và thay đổi sau khi kết thúc.
Loại B	Chỉ dùng thẻ từ	Người dùng nhận thẻ tại điểm bàn giao, sử dụng và trả thẻ sau khi kết thúc.
Loại C	Có cả mã số và thẻ; phiên sử dụng chọn mã	Áp dụng quy trình cấp mã, không mặc nhiên bàn giao thẻ.
Loại D	Có cả mã số và thẻ; phiên sử dụng chọn thẻ	Áp dụng quy trình nhận và trả thẻ, không mặc nhiên cấp mã.

Trong phần mô phỏng khóa số, nhóm dùng cấu hình SmartLock kết nối nền tảng

OneIoT. Ứng dụng khái quát phương thức tiếp cận thành khóa cơ/thẻ từ và khóa mã số trực tuyến. Mỗi yêu cầu dùng một phương thức theo loại khóa của phòng; quyền tạo mã được gắn với đúng phòng sau khi yêu cầu được duyệt.

2.1.2 Khảo sát các giải pháp tương tự

Để định vị giải pháp, nhóm đối chiếu RoomHub với các nhóm công cụ thường dùng để ghi lịch và đặt chỗ. Bảng 2.2 là so sánh định tính theo khả năng thiết kế, không phải phép đo hay đánh giá toàn bộ sản phẩm trên thị trường.

Bảng 2.2. So sánh các nhóm giải pháp quản lý đặt phòng

Tiêu chí	**Sổ ghi / bảng tính**	**Lịch tài nguyên trực tuyến**	**Phần mềm đặt chỗ thương mại**	**RoomHub**
Trạng thái phòng theo khung giờ	Thủ công	Có	Có	Có
Sơ đồ tầng, vị trí phòng	Không	Không	Thường có	Có (chữ U, 8 tầng)
Lọc sức chứa, thiết bị	Không	Hạn chế	Có	Có
Quy tắc riêng (hạn đặt trước, giới hạn yêu cầu, bảo trì)	Không	Hạn chế	Có, cấu hình theo gói	Có, quản trị viên chỉnh trong ứng dụng
Quy trình phê duyệt	Thủ công	Tự động chấp nhận hoặc từ chối	Có	Có, kèm đổi phòng
Cấp quyền tiếp cận theo loại khóa	Không	Không	Qua tích hợp riêng	Hẹn nhận khóa / mã PIN tạm thời
Chi phí, khả năng tùy biến	Công sức quản lý thủ công	Phụ thuộc gói dịch vụ	Phụ thuộc nhà cung cấp	Tự phát triển

Từ phân tích trên, nhóm ưu tiên sơ đồ tầng có trạng thái theo thời gian, thao tác đặt phòng ngay sau khi chọn phòng, kiểm tra quy tắc trước khi lưu, quy trình duyệt và đổi phòng có điều kiện, quyền tiếp cận theo loại khóa, cùng khả năng chỉnh tham số nghiệp vụ trong ứng dụng.

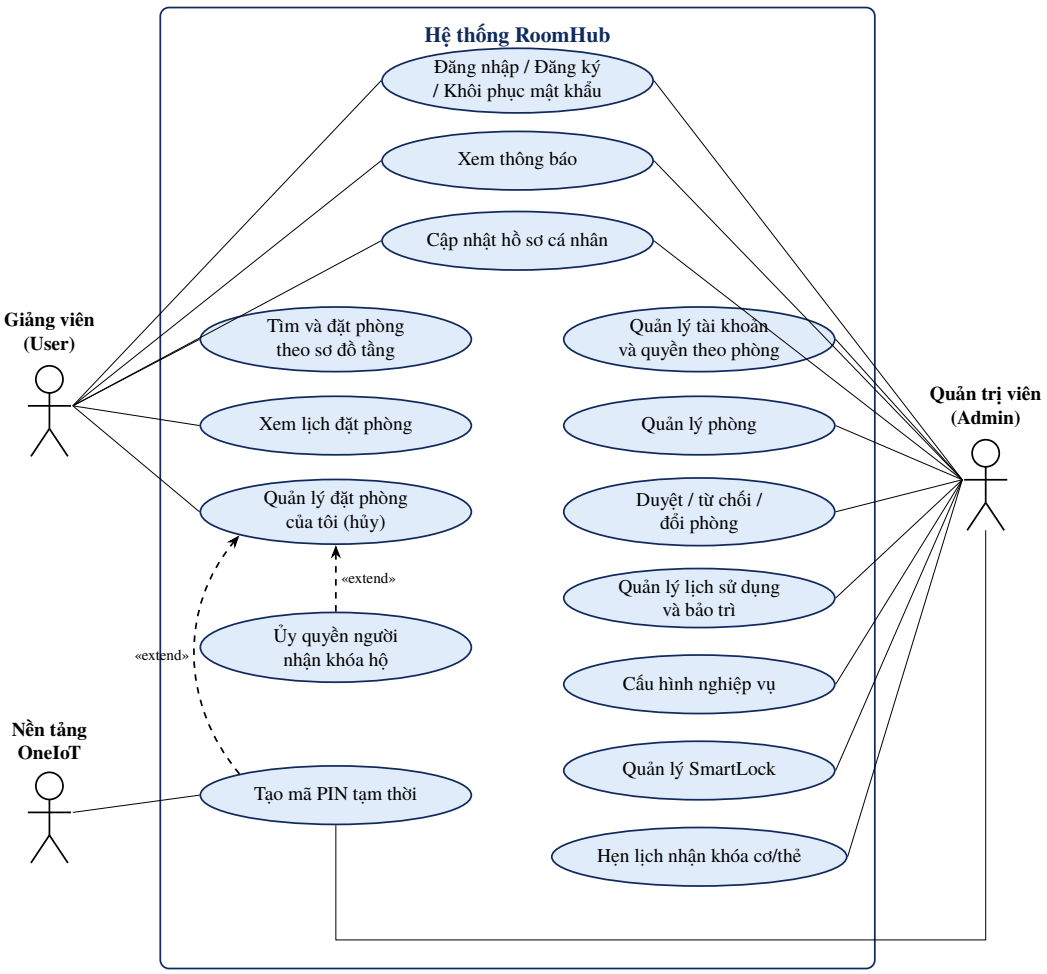
2.2 Tổng quan chức năng

Mục này tóm tắt chức năng của hệ thống ở mức tổng quan; đặc tả chi tiết được trình bày trong mục 2.3.

2.2.1 Biểu đồ use case tổng quát

Hệ thống có hai vai trò người dùng chính: giảng viên gửi và theo dõi yêu cầu, cán bộ quản lý duyệt và cấu hình. OneIoT và khóa thông minh là hệ thống ngoài trong nhánh cấp

mã. Hình 2.1 thể hiện use case ở mức khái niệm; phần truyền bản tin hiện được ứng dụng Android thực hiện trực tiếp. Vai trò được xác định từ tài khoản đăng nhập.

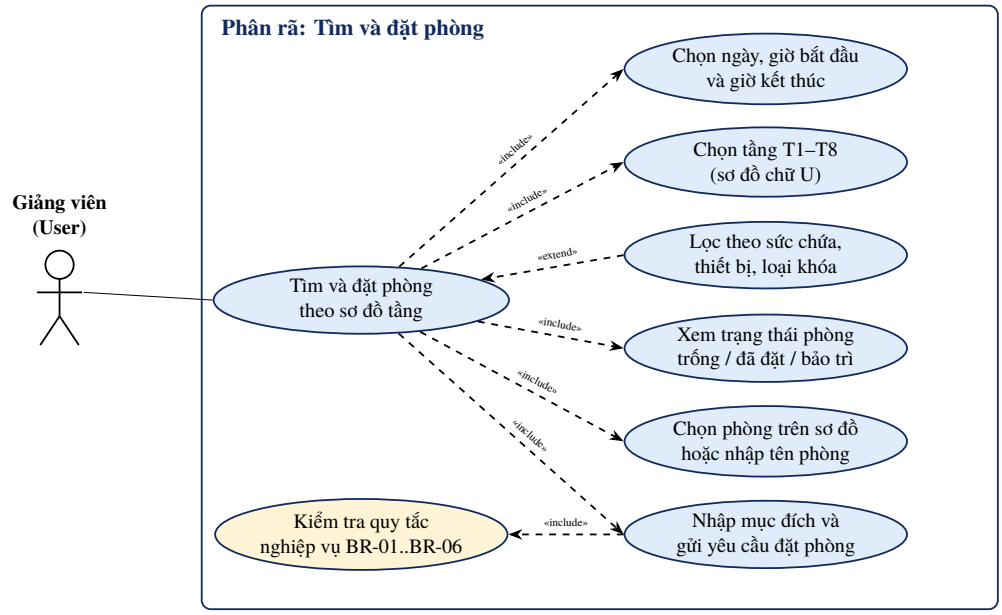


Hình 2.1. Biểu đồ use case tổng quát của hệ thống RoomHub

Các use case dùng chung gồm đăng nhập, đăng ký, khôi phục mật khẩu, xem thông báo và cập nhật hồ sơ. Người dùng có các chức năng tìm và đặt phòng, xem lịch theo trạng thái, quản lý yêu cầu của mình, ủy quyền người nhận khóa hộ và tạo mã PIN tạm thời khi được cấp quyền. Quản trị viên quản lý tài khoản, phòng, yêu cầu, lịch, cấu hình và khóa; thao tác duyệt, từ chối hoặc đổi phòng là phần trung tâm của quy trình.

2.2.2 Biểu đồ use case phân rã "Tìm và đặt phòng"

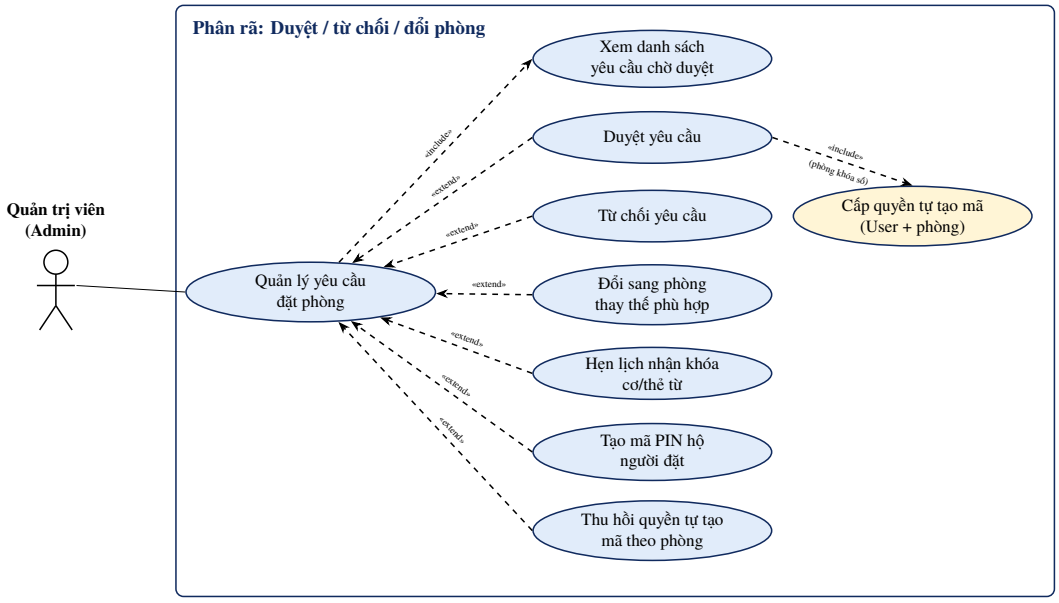
Use case "Tìm và đặt phòng" được phân rã như Hình 2.2. Người dùng bắt buộc chọn khung thời gian, chọn tầng, xem trạng thái phòng trên sơ đồ, chọn phòng và gửi yêu cầu; việc lọc theo sức chứa, thiết bị và loại khóa là tùy chọn. Khi gửi, hệ thống luôn thực hiện use case "Kiểm tra quy tắc nghiệp vụ" với các quy tắc trong Bảng 2.3.



Hình 2.2. Biểu đồ use case phân rã "Tìm và đặt phòng"

2.2.3 Biểu đồ use case phân rã "Duyệt / từ chối / đổi phòng"

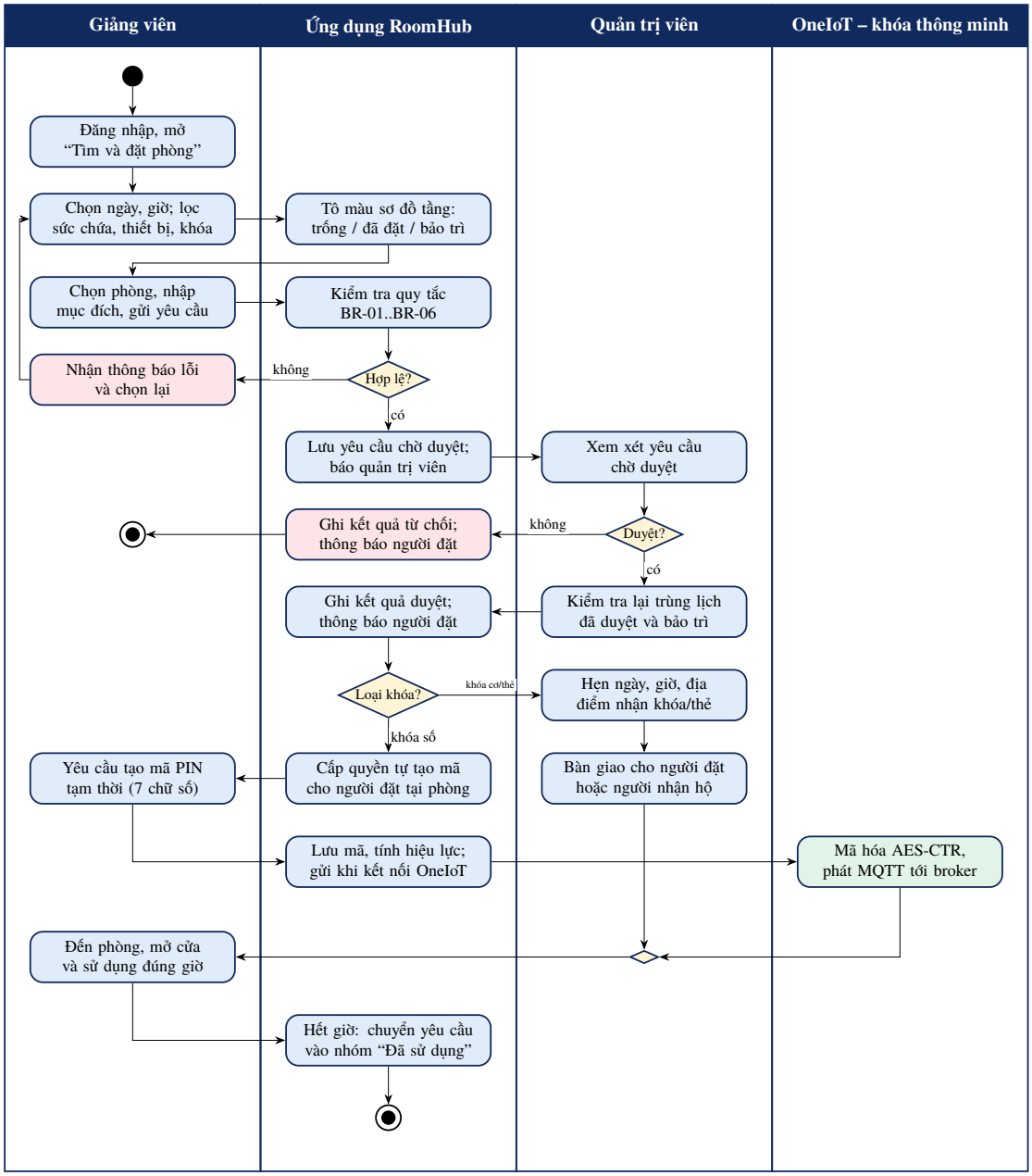
Hình 2.3 phân rã use case quản lý yêu cầu đặt phòng của quản trị viên. Quản trị viên xem các yêu cầu chờ, sau đó có thể duyệt, từ chối, đổi sang phòng phù hợp, hẹn lịch nhận khóa cơ hoặc thẻ, tạo mã PIN hoặc thu hồi quyền tự tạo mã. Khi duyệt yêu cầu tại phòng khóa số, hệ thống cấp quyền tự tạo mã cho đúng người đặt và phòng đó; quyền này không thay đổi vai trò tài khoản.



Hình 2.3. Biểu đồ use case phân rã "Duyệt / từ chối / đổi phòng"

2.2.4 Quy trình nghiệp vụ

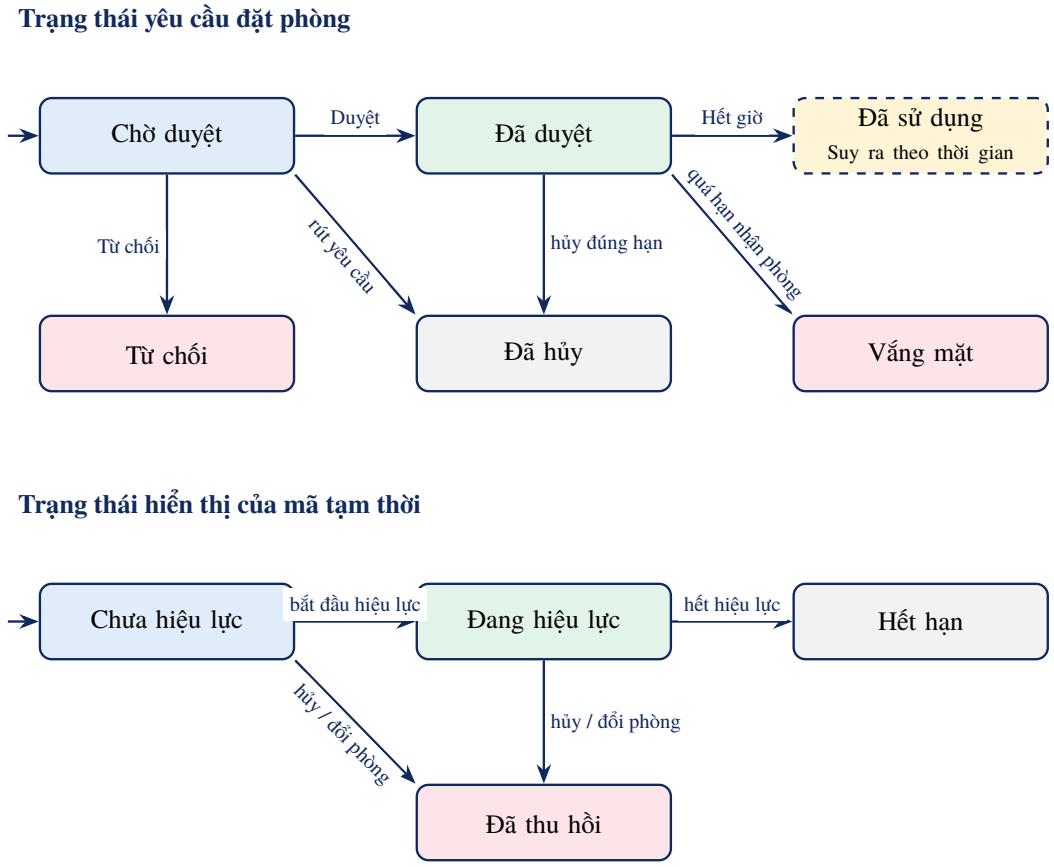
Quy trình đặt phòng kết hợp thao tác của hai vai trò như Hình 2.4. Người dùng chọn thời gian và phòng, nhập mục đích rồi gửi yêu cầu. Sau khi duyệt, phòng khóa số có thể tạo mã tạm thời; ứng dụng lưu mã trước và gửi bản tin qua OneIoT khi có kết nối. Với khóa cơ hoặc thẻ từ, hai bên thống nhất thời điểm nhận khóa và có thể ủy quyền người nhận hộ. Biểu đồ phản ánh luồng nghiệp vụ tổng quát; trạng thái phát bản tin không đồng nghĩa khóa đã nhận mã.



Hình 2.4. Biểu đồ hoạt động của quy trình đặt phòng và cấp quyền tiếp cận

Vòng đời yêu cầu được mô tả ở Hình 2.5. Bản triển khai lưu các trạng thái chờ duyệt, đã duyệt, từ chối, đã hủy và vắng mặt; thông tin check-in/check-out cùng phương thức tiếp cận được quản lý bằng các trường hoặc đối tượng liên quan. Trạng thái mã tạm thời hiển

thị theo thời gian và dấu thu hồi. Hình là sơ đồ khái niệm, nên không đồng nhất mọi bước sử dụng phòng với một trạng thái lưu trữ riêng.



Hình 2.5. Biểu đồ trạng thái yêu cầu đặt phòng và mã PIN tạm thời

2.3 Đặc tả chức năng

Mục này đặc tả năm use case quan trọng nhất. Các use case còn lại gồm hủy đặt phòng, hẹn lịch nhận khóa và ủy quyền, quản lý lịch bảo trì, cấu hình nghiệp vụ và quản lý SmartLock được đặc tả tại Phụ lục A. Trước hết, Bảng 2.3 liệt kê các quy tắc nghiệp vụ mà hệ thống đã hiện thực; các quy tắc được đánh mã để tham chiếu trong đặc tả và kiểm thử.

Bảng 2.3. Các quy tắc nghiệp vụ đã hiện thực trong RoomHub

Mã	Quy tắc	Tham số mặc định
BR-01	Phòng phải tồn tại và ở trạng thái có thể sử dụng.	–
BR-02	Ngày và giờ phải hợp lệ; giờ bắt đầu sớm hơn giờ kết thúc.	–
BR-03	Chỉ được đặt trong khoảng ngày cấu hình; có thể cho phép cùng ngày nếu giờ kết thúc chưa qua.	Mặc định 1 đến 3 ngày
BR-04	Không được đặt phòng trùng với lịch bảo trì chưa bị hủy của phòng đó.	–
BR-05	Không trùng với yêu cầu đang chờ hoặc đã duyệt khác của cùng phòng; một người không có hai yêu cầu chồng thời gian.	–
BR-06	Tổng số yêu cầu đang chờ hoặc sắp sử dụng của một người không vượt giới hạn; mục đích sử dụng không được để trống.	2 yêu cầu
BR-07	Chỉ duyệt yêu cầu chưa hết hiệu lực; kiểm tra lại bảo trì và trùng lịch.	–
BR-08	Người đặt chỉ hủy yêu cầu đã duyệt trước giờ bắt đầu một khoảng tối thiểu.	30 phút
BR-09	Quản trị viên chỉ đổi phòng trước giờ bắt đầu một khoảng tối thiểu; phòng thay thế phải cùng loại khóa, đủ sức chứa và thiết bị, không bảo trì và không trùng lịch.	30 phút
BR-10	Mã PIN chỉ tạo cho yêu cầu đã duyệt tại phòng khóa số đang gắn khóa thông minh; hiệu lực có khoảng đệm quanh giờ sử dụng và bị thu hồi khi hủy hoặc đổi phòng.	10 phút
BR-11	Người đặt chỉ tự tạo mã khi có quyền còn hiệu lực cho đúng phòng; quản trị viên có thể thu hồi quyền theo phòng.	–
BR-12	Ủy quyền nhận hộ và hẹn nhận khóa chỉ áp dụng cho yêu cầu đã duyệt tại phòng dùng khóa cơ hoặc thẻ; lịch hẹn phải trước giờ sử dụng.	–

2.3.1 Đặc tả use case "Đăng nhập"

Bảng 2.4. Đặc tả use case UC01 – Đăng nhập

Mã use case	UC01
Tác nhân	Người dùng, quản trị viên
Mô tả	Người dùng xác thực bằng tên đăng nhập và mật khẩu; hệ thống xác định vai trò và mở giao diện tương ứng.
Tiền điều kiện	Tài khoản đã tồn tại và đang ở trạng thái hoạt động.
Luồng chính	1. Người dùng mở ứng dụng, nhập tài khoản và mật khẩu, bấm "Đăng nhập". 2. Hệ thống chuẩn hóa tên đăng nhập (bỏ khoảng trắng, chuyển chữ thường). 3. Hệ thống xác thực tài khoản đang hoạt động; ở chế độ máy chủ, dịch vụ Java tạo phiên đăng nhập. 4. Hệ thống mở "Chế độ quản trị viên" hoặc "Chế độ người dùng" theo vai trò tài khoản.
Luồng phát sinh	3a. Sai thông tin hoặc tài khoản bị khóa: hiển thị lỗi và giữ nguyên màn hình. 3b. Người dùng chọn "Đăng ký" hoặc "Quên mật khẩu?" để tạo tài khoản mới hoặc đặt lại mật khẩu bằng mã khôi phục.
Hậu điều kiện	Phiên đăng nhập được thiết lập; mọi yêu cầu tiếp theo tới máy chủ mang token Bearer.

2.3.2 Đặc tả use case "Tìm và đặt phòng"

Bảng 2.5. Đặc tả use case UC02 – Tìm và đặt phòng

Mã use case	UC02
Tác nhân	Người dùng
Tiền điều kiện	Người dùng đã đăng nhập và tài khoản còn hoạt động.
Luồng chính	1. Người dùng mở "Tìm và đặt phòng"; hệ thống sinh danh sách ngày được phép theo cấu hình (mặc định từ ngày mai đến ba ngày sau). 2. Người dùng chọn ngày, giờ bắt đầu, giờ kết thúc và tầng từ 1 đến 8. 3. Người dùng có thể lọc theo sức chứa tối thiểu, thiết bị cần có và loại khóa. 4. Hệ thống hiển thị bảy phòng của tầng trên sơ đồ chữ U, mỗi ô ghi sức chứa, thiết bị và tô màu trống, đã đặt, bảo trì hoặc không khớp lọc; đồng thời báo số phòng phù hợp đang trống. 5. Người dùng chọn một phòng trống (hoặc nhập tên phòng để tìm), nhập mục đích và gửi yêu cầu đặt phòng. 6. Hệ thống kiểm tra BR-01 đến BR-06, lưu yêu cầu chờ duyệt, thông báo cho quản trị viên và làm mới sơ đồ.
Luồng phát sinh	5a. Phòng được chọn không ở trạng thái trống: yêu cầu chọn phòng khác. 6a. Vi phạm quy tắc: hiển thị lý do và không lưu yêu cầu.
Hậu điều kiện	Yêu cầu chờ duyệt được lưu; phòng hiển thị là đã có yêu cầu trong khung giờ đó; quản trị viên nhận thông báo.

2.3.3 Đặc tả use case "Duyệt yêu cầu đặt phòng"

Bảng 2.6. Đặc tả use case UC03 – Duyệt yêu cầu đặt phòng

Mã use case	UC03
Tác nhân	Quản trị viên
Tiền điều kiện	Quản trị viên đã đăng nhập; có yêu cầu đang chờ duyệt.
Luồng chính	1. Quản trị viên mở danh sách yêu cầu đặt phòng hoặc thông báo về yêu cầu mới. 2. Hệ thống hiển thị thông tin người đặt, phòng, ngày giờ, mục đích. 3. Quản trị viên chọn duyệt. 4. Hệ thống khóa bản ghi yêu cầu, kiểm tra BR-07: yêu cầu còn chờ, chưa qua giờ kết thúc, không trùng bảo trì hay yêu cầu đã duyệt khác. 5. Hệ thống chuyển sang trạng thái đã duyệt, ghi người và thời điểm xử lý. 6. Nếu phòng dùng khóa mã số, hệ thống cấp quyền tự tạo mã cho người đặt tại phòng đó. 7. Hệ thống gửi thông báo "Yêu cầu đã được duyệt" cho người đặt.
Luồng phát sinh	3a. Quản trị viên chọn từ chối: hệ thống lưu kết quả và thông báo cho người đặt. 4a. Vi phạm BR-07: hiển thị lý do, trạng thái không đổi.
Hậu điều kiện	Yêu cầu được duyệt hoặc bị từ chối; người đặt nhận thông báo. Với phòng khóa số, quyền tự tạo mã theo phòng được cấp khi duyệt.

2.3.4 Đặc tả use case "Đổi phòng thay thế"

Bảng 2.7. Đặc tả use case UC04 – Đổi phòng thay thế

Mã use case	UC04
Tác nhân	Quản trị viên
Tiền điều kiện	Yêu cầu đã được duyệt và còn ít nhất 30 phút (theo cấu hình) trước giờ bắt đầu.
Luồng chính	1. Quản trị viên chọn yêu cầu cần đổi, ví dụ khi phòng cũ phải phục vụ sự kiện ưu tiên. 2. Hệ thống chỉ đề xuất phòng khác đang khả dụng, cùng loại khóa, có sức chứa và thiết bị không kém phòng cũ. 3. Quản trị viên chọn phòng thay thế và nhập lý do. 4. Hệ thống kiểm tra BR-09 cùng lịch bảo trì và các yêu cầu chờ/đã duyệt của phòng mới. 5. Hệ thống cập nhật phòng, ghi lịch sử đổi phòng, thu hồi mã PIN cũ và lịch hẹn nhận khóa cũ; nếu phòng mới dùng khóa số thì cấp quyền tạo mã cho phòng mới. 6. Hệ thống thông báo cho người đặt kèm lý do đổi phòng.
Luồng phát sinh	4a. Phòng thay thế khác loại khóa: báo "Phòng thay thế phải có cùng loại khóa với phòng hiện tại". 4b. Quá hạn đổi phòng hoặc thiếu lý do: báo lỗi tương ứng.
Hậu điều kiện	Yêu cầu được chuyển sang phòng mới và ghi lịch sử đổi phòng; quyền tiếp cận phòng cũ bị vô hiệu.

2.3.5 Đặc tả use case "Tạo mã PIN tạm thời"

Bảng 2.8. Đặc tả use case UC05 – Tạo mã PIN tạm thời

Mã use case	UC05
Tác nhân	Người đặt hoặc quản trị viên; nền tảng OneIoT
Tiền điều kiện	Yêu cầu đã duyệt tại phòng khóa số đang gắn khóa thông minh; người đặt có quyền tự tạo mã theo phòng. Có thể tạo mã trong ứng dụng khi chưa kết nối OneIoT.
Luồng chính	1. Người đặt mở danh sách yêu cầu của mình, chọn yêu cầu và tạo mã. 2. Hệ thống kiểm tra BR-10, BR-11 và bảo đảm yêu cầu chưa có mã đang hiệu lực. 3. Hệ thống sinh mã bảy chữ số, tính thời điểm hiệu lực từ giờ bắt đầu trừ 10 phút đến giờ kết thúc cộng 10 phút theo cấu hình mặc định. 4. Hệ thống cấp vị trí mật khẩu của khóa và lưu mã cùng khoảng hiệu lực vào yêu cầu. 5. Nếu OneIoT đang kết nối, ứng dụng mã hóa và phát bản tin tạo mã qua MQTT; nếu chưa kết nối, mã chờ gửi trong ứng dụng. 6. Hệ thống hiển thị mã và thông báo rõ trạng thái đã gửi hoặc chờ kết nối.
Luồng phát sinh	2a. Quyền đã bị thu hồi: báo "Quyền tự tạo mã tại phòng này đã bị thu hồi hoặc chưa được cấp". 4a. Phòng chưa gắn khóa: báo "Phòng chưa được gắn với SmartLock đang quản lý". 5a. Chưa kết nối hoặc gửi lỗi: mã vẫn được lưu để thử gửi khi kết nối lại; mã hết hiệu lực hoặc đã thu hồi sẽ không gửi.
Hậu điều kiện	Ứng dụng lưu mã và trạng thái gửi; việc khóa thật áp dụng mã chỉ được xác nhận bằng phản hồi hoặc kiểm thử thiết bị.

2.4 Yêu cầu phi chức năng

Về **tính đúng đắn và độ tin cậy**, hệ thống kiểm tra xung đột theo khoảng thời gian nửa mở, lịch bảo trì và quyền của người gọi. Ở chế độ máy chủ, thao tác ghi được kiểm tra trong giao dịch có khóa; tính đúng đắn khi nhiều thiết bị truy cập đồng thời cần được tiếp tục kiểm thử với máy chủ triển khai thực tế.

Về **bảo mật**, các API cần phiên đăng nhập hợp lệ theo từng loại thao tác. Token OneIoT được nhập cho phiên kết nối và không nằm trong kho mã nguồn. Ứng dụng mã hóa mật khẩu trước khi phát bản tin MQTT; phần lưu trữ mã tại ứng dụng và máy chủ cần được gia cố trước triển khai thực tế.

Về **tính khả dụng**, giao diện tiếng Việt và điều khiển chọn ngày giờ phù hợp thao tác cảm ứng trên điện thoại Android. Bản Release phải khởi động độc lập với Metro; chế độ dùng máy chủ còn phụ thuộc kết nối tới dịch vụ đang chạy.

Về **tính dễ bảo trì và mở rộng**, mã nguồn được chia theo module nghiệp vụ, các tham số chính sách được lưu dưới dạng cấu hình thay vì hằng số trong giao diện, và việc chuyển giữa lưu cục bộ với đồng bộ máy chủ chỉ phụ thuộc một cờ cấu hình.

Về **yêu cầu kỹ thuật của bản triển khai**, máy chủ dùng Java 17, Spring Boot,

JPA/Hibernate và H2 trong môi trường thử nghiệm; ứng dụng dùng React Native, TypeScript, Async Storage và module Android viết bằng Kotlin để giao tiếp OneIoT. Đây là lựa chọn triển khai của nhóm, không phải tập công nghệ được suy diễn là bắt buộc của mọi học phần.

Chương này đã xác định vai trò, các use case, quy tắc nghiệp vụ và yêu cầu phi chức năng. Các yêu cầu về giao dịch, phân quyền, dữ liệu cục bộ và bản tin khóa là cơ sở cho lựa chọn công nghệ ở Chương 3.

CHƯƠNG 3. NỀN TẢNG LÝ THUYẾT VÀ CÔNG NGHỆ SỬ DỤNG

Chương 2 đã xác định các yêu cầu chức năng và phi chức năng của RoomHub, trong đó nổi bật là kiểm tra quyền và quy tắc nghiệp vụ ở phía máy chủ, chống trùng lịch khi có truy cập đồng thời, chạy trên điện thoại Android và tích hợp khóa thông minh. Chương này tóm lược nền tảng lý thuyết và công nghệ được dùng để đáp ứng từng nhóm yêu cầu, gồm lập trình hướng đối tượng với Java, khung Spring Boot, JPA và cơ sở dữ liệu H2, kiến trúc REST, React Native với TypeScript, giao thức MQTT cùng mã hóa AES, và các công cụ mô hình hóa, kiểm thử. Với mỗi lựa chọn, báo cáo nêu phương án thay thế và lý do quyết định.

3.1 Lập trình hướng đối tượng với Java 17

Lập trình hướng đối tượng tổ chức chương trình thành các đối tượng gồm dữ liệu và hành vi, dựa trên các nguyên lý đóng gói, trừu tượng hóa, kế thừa và đa hình [2]. Java là ngôn ngữ có kiểu tĩnh, chạy trên máy ảo JVM và được đặc tả trong *Java Language Specification* [3]. Phiên bản 17 hỗ trợ các lớp dữ liệu bất biến và cú pháp ngắn gọn, phù hợp với việc mô hình hóa thông tin trao đổi giữa các tầng.

Trong dịch vụ Java của RoomHub, tầng nghiệp vụ đóng gói các phép kiểm tra lịch, bảo trì và phân quyền; tầng giao tiếp chỉ gọi các thao tác công khai tương ứng với ca sử dụng. Bộ lọc phiên kế thừa thành phần xử lý yêu cầu của Spring, còn bộ chuyển đổi thiết bị triển khai giao diện ánh xạ dữ liệu. Các loại lỗi xác thực và phân quyền kế thừa cùng một loại ngoại lệ, cho phép bộ xử lý tập trung chọn phản hồi HTTP phù hợp. Thông tin đầu vào và kết quả được mô hình hóa bằng các đối tượng dữ liệu bất biến; cách tổ chức này hạn chế sửa đổi ngoài ý muốn giữa các tầng [4].

Phương án thay thế là C# trên nền .NET hoặc Node.js với TypeScript ở phía máy chủ. Nhóm chọn Java để đáp ứng định hướng thực hiện bài tập lớn bằng Java và vì hệ sinh thái Java có sẵn khung Spring và đặc tả JPA đã chuẩn hóa, giúp nhóm tập trung vào mô hình nghiệp vụ.

3.2 Spring Boot và cơ chế đảo ngược điều khiển

Spring Framework cung cấp cơ chế đảo ngược điều khiển để quản lý vòng đời đối tượng và truyền các phụ thuộc qua hàm khởi tạo. Spring Boot bổ sung tự cấu hình, máy chủ web nhúng và các gói thư viện theo mục đích sử dụng [5]. Nhờ đó, tầng giao tiếp, tầng nghiệp vụ và tầng dữ liệu có thể được lắp ghép qua cấu hình của khung ứng dụng.

RoomHub dùng tiêm phụ thuộc để lớp giao tiếp gọi lớp nghiệp vụ, còn lớp nghiệp vụ truy cập dữ liệu qua JPA mà không tự tạo các thành phần này. Bộ lọc xác thực kiểm tra

phiên trước khi xử lý các yêu cầu cần đăng nhập. Lỗi nghiệp vụ được chuyển thành phản hồi HTTP tại một điểm chung. Những thao tác ghi dữ liệu chạy trong giao dịch để các bước kiểm tra và cập nhật được xử lý cùng nhau.

Các lựa chọn thay thế gồm Jakarta EE thuần với Servlet hoặc các khung nhẹ như Micronaut, Quarkus. Nhóm chọn Spring Boot 3.4.4 vì tài liệu phong phú, cấu hình tối thiểu và hỗ trợ đầy đủ Java 17.

3.3 Jakarta Persistence, Hibernate và cơ sở dữ liệu H2

Jakarta Persistence (JPA) là đặc tả ánh xạ đối tượng – quan hệ của Java, cho phép biểu diễn bảng dữ liệu bằng lớp thực thể và truy vấn qua mô hình đối tượng [6]. Hibernate hiện thực JPA trong dự án. Cơ chế quản lý thực thể theo dõi các thay đổi trong một giao dịch rồi ghi xuống cơ sở dữ liệu khi giao dịch hoàn tất [7].

JPA hỗ trợ nhiều cách điều khiển truy cập đồng thời. Khóa lạc quan phát hiện xung đột khi ghi, còn khóa bi quan giữ quyền ghi trên dữ liệu được chọn đến khi giao dịch kết thúc [8]. Với bài toán đặt phòng, hai giao dịch có thể cùng thấy lịch trống trước khi thêm hai yêu cầu chồng giờ. RoomHub vì vậy khóa bản ghi cấu hình và phòng trước khi kiểm tra lịch rồi ghi yêu cầu (xem mục 5.1).

H2 là hệ quản trị cơ sở dữ liệu quan hệ viết bằng Java, chạy nhúng trong tiến trình ứng dụng và hỗ trợ chế độ lưu tệp [9]. RoomHub dùng H2 ở chế độ tệp để lưu dữ liệu thử nghiệm; Hibernate tạo và cập nhật lược đồ từ các lớp thực thể. So với MySQL hoặc PostgreSQL, H2 không cần cài đặt máy chủ riêng nên phù hợp giai đoạn phát triển và demo. Khi chuyển sang PostgreSQL vẫn cần kiểm tra lại lược đồ, giao dịch, truy vấn và quá trình di chuyển dữ liệu.

3.4 Kiến trúc REST và định dạng JSON

REST (Representational State Transfer) là kiểu kiến trúc do Fielding đề xuất, trong đó tài nguyên được định danh bằng URI và thao tác qua các phương thức HTTP [10]. RoomHub trao đổi dữ liệu dạng JSON cho các nhóm chức năng tài khoản, phòng, đặt phòng, bảo trì, cấu hình và thông báo. Phiên đăng nhập được nhận diện bằng mã phiên gửi kèm yêu cầu. Danh sách điểm cuối tiêu biểu nằm ở Phụ lục B.

Các phương án thay thế là GraphQL, phù hợp khi máy khách cần truy vấn linh hoạt nhiều dạng dữ liệu, và gRPC, phù hợp giao tiếp giữa các dịch vụ nội bộ. Với phạm vi hiện tại, REST thuận tiện để triển khai và kiểm thử các ca sử dụng của ứng dụng di động.

3.5 React Native và TypeScript

React Native là khung phát triển ứng dụng di động đa nền tảng dựa trên React; giao diện được mô tả bằng các thành phần khai báo và hiển thị qua thành phần gốc của nền tảng [11]. TypeScript bổ sung kiểm tra kiểu tĩnh cho JavaScript, giúp nhận diện sớm sự

không khớp giữa các trạng thái yêu cầu và dữ liệu truyền qua các màn hình [12]. Ứng dụng lưu dữ liệu cục bộ bằng Async Storage [13]; thành phần Android viết bằng Kotlin hỗ trợ thông báo hệ thống và giao tiếp khóa.

Các phương án thay thế gồm Flutter với ngôn ngữ Dart và ứng dụng Android gốc bằng Kotlin. Nhóm chọn React Native với TypeScript để dùng chung phần lớn mã giao diện và kiểm tra kiểu dữ liệu trong ứng dụng. Bản được xây dựng và kiểm thử trong đề tài hiện chạy trên Android; khả năng chạy trên iOS chưa được kiểm chứng.

3.6 Giao thức MQTT, chuẩn oneM2M và mã hóa AES

MQTT là giao thức xuất bản – đăng ký qua broker, phù hợp truyền lệnh tới thiết bị IoT [14]. OneIoT sử dụng quy ước chủ đề và nội dung bản tin theo oneM2M [15]. Trong bản ứng dụng hiện tại, thành phần Android kết nối broker qua TLS sau khi người dùng nhập thông tin xác thực cho phiên sử dụng. Lệnh tạo mã được đóng gói theo định dạng thiết bị; broker nhận bản tin chưa có nghĩa khóa đã thực thi.

Mật khẩu tạm thời không được gửi dạng rõ mà mã hóa bằng AES ở chế độ CTR. Chế độ CTR biến mã khối thành mã dòng bằng cách mã hóa một bộ đếm rồi XOR với bản rõ, nên độ dài bản mã bằng độ dài bản rõ và không cần đệm [16]. Khóa 256 bit được dẫn xuất bằng SHA-256 từ định danh thiết bị, còn giá trị khởi tạo được dẫn xuất từ khoảng hiệu lực và số hiệu mật khẩu theo định dạng mà firmware khóa DLWA12 yêu cầu.

Một tiến trình gateway Python cũng có trong kho mã nguồn để tham khảo và thử nghiệm giao thức, nhưng luồng phát lệnh của ứng dụng hiện dùng module Android trực tiếp. Cách này giúp người dùng tạo mã trong ứng dụng và gửi khi kết nối OneIoT, đồng thời yêu cầu quản lý cẩn thận token chỉ trong phiên và phản hồi từ khóa.

3.7 Mô hình hóa UML và kiểm thử tự động

Các biểu đồ use case, hoạt động, trạng thái, gói, lớp và trình tự trong báo cáo sử dụng ký pháp UML 2.5.1 [17]. Kiểm thử tự động phía ứng dụng dùng Jest [18]; các ca kiểm thử gọi tầng nghiệp vụ trên kho dữ liệu giả lập để kiểm tra quy tắc mà không cần mở giao diện. Nhóm còn dùng công cụ phân tích tĩnh và trình biên dịch TypeScript để kiểm tra mã nguồn.

Bảng 3.1 tổng hợp liên hệ giữa yêu cầu, công nghệ được chọn và phương án thay thế đã cân nhắc.

Bảng 3.1. Tổng hợp lựa chọn công nghệ theo yêu cầu

Yêu cầu (Chương 2)	**Công nghệ chọn**	**Phương án thay thế**
Mô hình nghiệp vụ hướng đối tượng, kiểm tra quyền phía máy chủ	Java 17, Spring Boot 3.4.4	C#/.NET, Node.js, Jakarta EE thuần
Lưu trữ, chống trùng lịch đồng thời	JPA/Hibernate, khóa bi quan, H2	JDBC thủ công, MyBatis; PostgreSQL, MySQL
Giao tiếp ứng dụng – máy chủ	REST/JSON, token Bearer	GraphQL, gRPC
Ứng dụng trên điện thoại Android	React Native 0.86.2, TypeScript 5.8	Flutter, Kotlin gốc
Lưu cục bộ khi chưa có máy chủ	Async Storage	SQLite, MMKV
Gửi mật khẩu tạm thời tới khóa	Module Android, MQTT/oneM2M, AES-CTR	BLE trực tiếp, máy chủ Java gọi broker
Kiểm thử quy tắc nghiệp vụ	Jest, JUnit, ESLint, TypeScript compiler	Kiểm thử thủ công

Chương này đã trình bày cơ sở lý thuyết và lý do lựa chọn từng công nghệ. Java cùng Spring Boot và JPA đáp ứng yêu cầu mô hình hóa nghiệp vụ và kiểm soát giao dịch ở máy chủ; React Native với TypeScript đáp ứng yêu cầu chạy trên điện thoại; MQTT và AES-CTR phục vụ tích hợp khóa thông minh. Chương 4 sẽ trình bày cách các công nghệ này được tổ chức thành kiến trúc và thiết kế chi tiết của hệ thống.

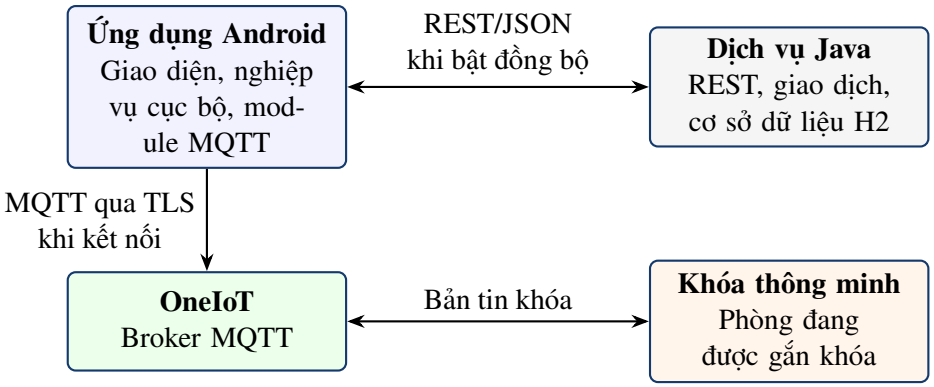
CHƯƠNG 4. PHÂN TÍCH THIẾT KẾ, TRIỂN KHAI VÀ ĐÁNH GIÁ HỆ THỐNG

Chương 3 đã trình bày các công nghệ được lựa chọn. Chương này mô tả cách tổ chức các công nghệ đó thành hệ thống RoomHub, đi từ thiết kế kiến trúc, thiết kế gói, thiết kế giao diện, thiết kế lớp và cơ sở dữ liệu tới kết quả xây dựng, kiểm thử và triển khai trên thiết bị thật.

4.1 Thiết kế kiến trúc

4.1.1 Lựa chọn kiến trúc phần mềm

RoomHub trong phạm vi báo cáo gồm ứng dụng Android, backend Java, cơ sở dữ liệu và nhánh tích hợp khóa OneIoT như Hình 4.1. Ứng dụng hiện bật chế độ remote và gọi REST/JSON tới backend; backend xử lý nghiệp vụ và lưu dữ liệu vào H2 qua JPA. Các API cho tài khoản, phòng, booking, bảo trì, cấu hình, quyền PIN và thông báo đã được triển khai. Khi dùng nhiều thiết bị cùng server, dữ liệu được đọc/ghi qua kho dữ liệu chung ở phía server. Chế độ local Async Storage là lựa chọn riêng, không tự đồng bộ ngược lên server khi offline. Với phòng khóa số, ứng dụng gửi MQTT trực tiếp qua module Android sau khi người dùng kết nối OneIoT trong phiên. Gateway Python chỉ phục vụ thử nghiệm giao thức.



Hình 4.1. Kiến trúc triển khai của RoomHub theo mã nguồn hiện tại

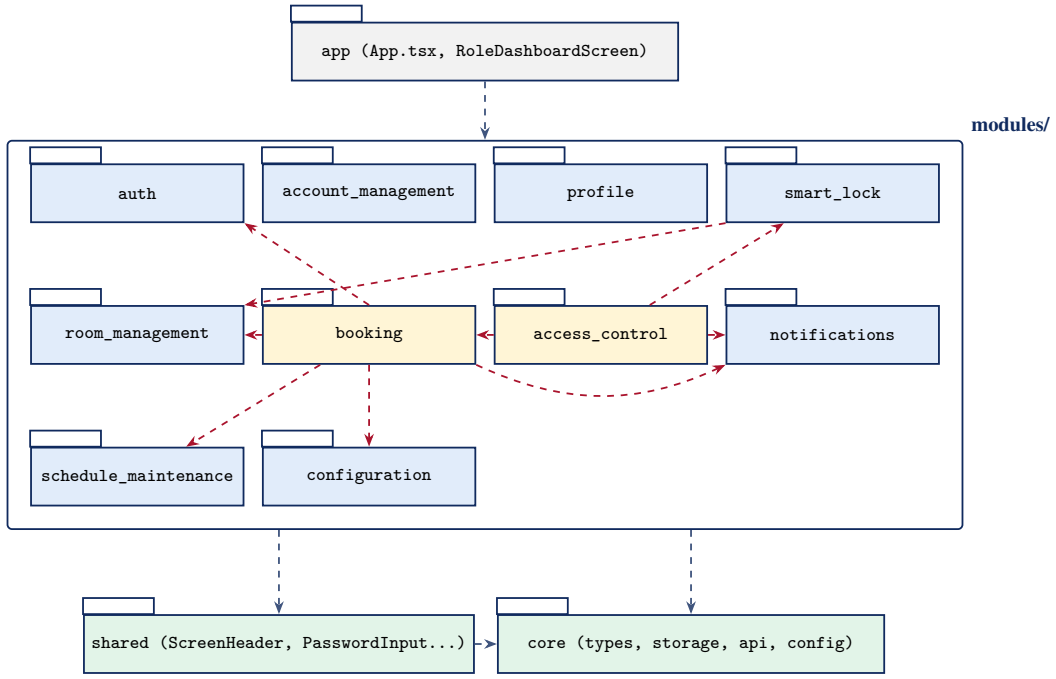
Bên trong mỗi thành phần, nhóm áp dụng kiến trúc phân tầng để tách xử lý giao tiếp, nghiệp vụ và dữ liệu [19]. Ở dịch vụ Java, bộ lọc xác thực kiểm tra phiên, tầng giao tiếp tiếp nhận yêu cầu, tầng nghiệp vụ áp dụng các quy tắc đặt phòng, còn tầng dữ liệu quản lý thực thể qua JPA [7]. Tầng giao tiếp chuyển yêu cầu hợp lệ cho tầng nghiệp vụ; bộ xử lý lỗi tập trung chuyển ngoại lệ thành phản hồi HTTP. Cách phân chia này giúp kiểm thử các quy tắc mà không cần gắn chúng vào mã xử lý HTTP.

Ở ứng dụng di động, mỗi module nghiệp vụ có giao diện, dịch vụ và mô hình dữ liệu. Tầng lõi cung cấp lưu trữ cục bộ, máy khách API và cờ cấu hình. Giao diện gọi tầng dịch

vụ; tầng này chọn nguồn dữ liệu theo chế độ chạy. Chi tiết được trình bày tại mục 5.3.

4.1.2 Thiết kế tổng quan

Hình 4.2 là biểu đồ gói của ứng dụng di động. Phần điều hướng kết nối các màn hình theo vai trò; mười module nghiệp vụ dùng chung thành phần giao diện và các dịch vụ nền như lưu trữ, xác thực và giao tiếp API. Chức năng đặt phòng lấy thông tin phòng, bảo trì và cấu hình để kiểm tra điều kiện. Chức năng cấp mã dùng thông tin yêu cầu đã duyệt và phòng đang gắn khóa. Mũi tên trong biểu đồ thể hiện chiều phụ thuộc giữa các nhóm chức năng.



Hình 4.2. Biểu đồ gói của ứng dụng di động RoomHub

Giữa đặt phòng và quản lý quyền tạo mã vẫn tồn tại phụ thuộc hai chiều: duyệt yêu cầu có thể cấp quyền, còn tạo mã cần đối chiếu yêu cầu đã duyệt. Bản hiện tại giới hạn phụ thuộc này ở tầng dữ liệu thay vì để hai phần giao diện gọi lẫn nhau. Đây là điểm cần tiếp tục đơn giản hóa khi mở rộng hệ thống. Bảng 4.1 tóm tắt nhiệm vụ của từng module.

Bảng 4.1. Nhiệm vụ các module nghiệp vụ của ứng dụng di động

Nhóm chức năng	**Nhiệm vụ**	**Vai trò**
Xác thực	Đăng ký, đăng nhập, đăng xuất, đổi và khôi phục mật khẩu	Chung
Quản lý tài khoản	Thêm, khóa/mở, đặt lại mật khẩu, xóa tài khoản; thu hồi quyền tạo mã theo phòng	Quản trị viên
Quản lý phòng	Tạo, sửa, xóa phòng; quản lý tầng, sức chứa, thiết bị, loại khóa; hỗ trợ tìm và đặt phòng	Chung
Đặt phòng	Tạo, duyệt, từ chối, hủy, đổi phòng; hẹn nhận khóa, ủy quyền và theo dõi lịch	Chung
Quyền tạo mã	Cấp, thu hồi quyền theo người dùng và phòng; tạo và tính hiệu lực mã PIN	Chung
Khóa thông minh	Gắn hoặc tháo khóa khỏi phòng, kết nối OneIoT theo phiên và gửi lệnh	Quản trị viên
Lịch và bảo trì	Theo dõi lịch sử dụng; tạo, hủy bảo trì; kiểm tra xung đột	Quản trị viên
Cấu hình	Điều chỉnh cửa sổ đặt trước, giới hạn yêu cầu, hạn hủy hoặc đổi phòng và thông báo	Quản trị viên
Thông báo	Hiển thị thông báo, số chưa đọc và thông báo hệ thống Android	Chung
Hồ sơ	Xem và sửa họ tên, email, số điện thoại, đơn vị	Chung

4.1.3 Thiết kế chi tiết gói

Dịch vụ Java tách phần khởi động, xác thực phiên, tiếp nhận API, xử lý lỗi, nghiệp vụ, mô hình dữ liệu và dữ liệu mẫu. Dữ liệu khởi tạo phục vụ trình diễn gồm tài khoản mẫu, 56 phòng trên tám tầng và cấu hình mặc định. Hình 4.4 khái quát trách nhiệm và quan hệ giữa các nhóm lớp chính.

Bộ lọc phiên và bộ tiếp nhận API cùng gọi tầng nghiệp vụ; tầng này làm việc với đối tượng trao đổi dữ liệu và các loại lỗi nghiệp vụ. Dữ liệu mã PIN được lưu gắn với yêu cầu đặt phòng thay vì quản lý như một yêu cầu độc lập. Các liên hệ giữa yêu cầu, phòng và tài khoản được biểu diễn bằng định danh để thuận tiện trao đổi với ứng dụng di động.

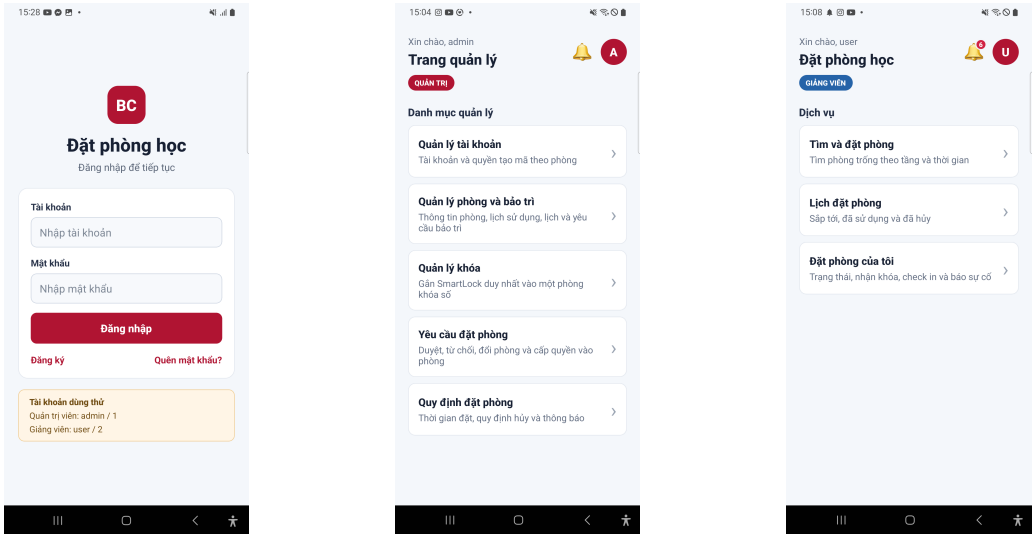
4.2 Thiết kế chi tiết

4.2.1 Thiết kế giao diện

Giao diện được thiết kế cho điện thoại Android theo chiều dọc. Màu đỏ nhấn các thao tác chính, màu xanh và xám phân biệt trạng thái phòng; thông báo thành công dùng màu tích cực, lỗi dùng màu cảnh báo. Thành phần đầu trang, nút quay lại, chuông thông báo và menu tài khoản được dùng nhất quán giữa các màn hình. Người dùng chọn ngày giờ bằng điều khiển cảm ứng và nhận phản hồi ngay khi gửi thao tác.

Sau khi đăng nhập, ứng dụng mở bảng điều khiển tương ứng vai trò. Chuông thông báo và menu tài khoản nằm ở vùng đầu trang; các chức năng nghiệp vụ được nhóm thành

thẻ để tránh danh sách dài. Hình 4.3 dùng ảnh chụp từ bản ứng dụng hiện tại.

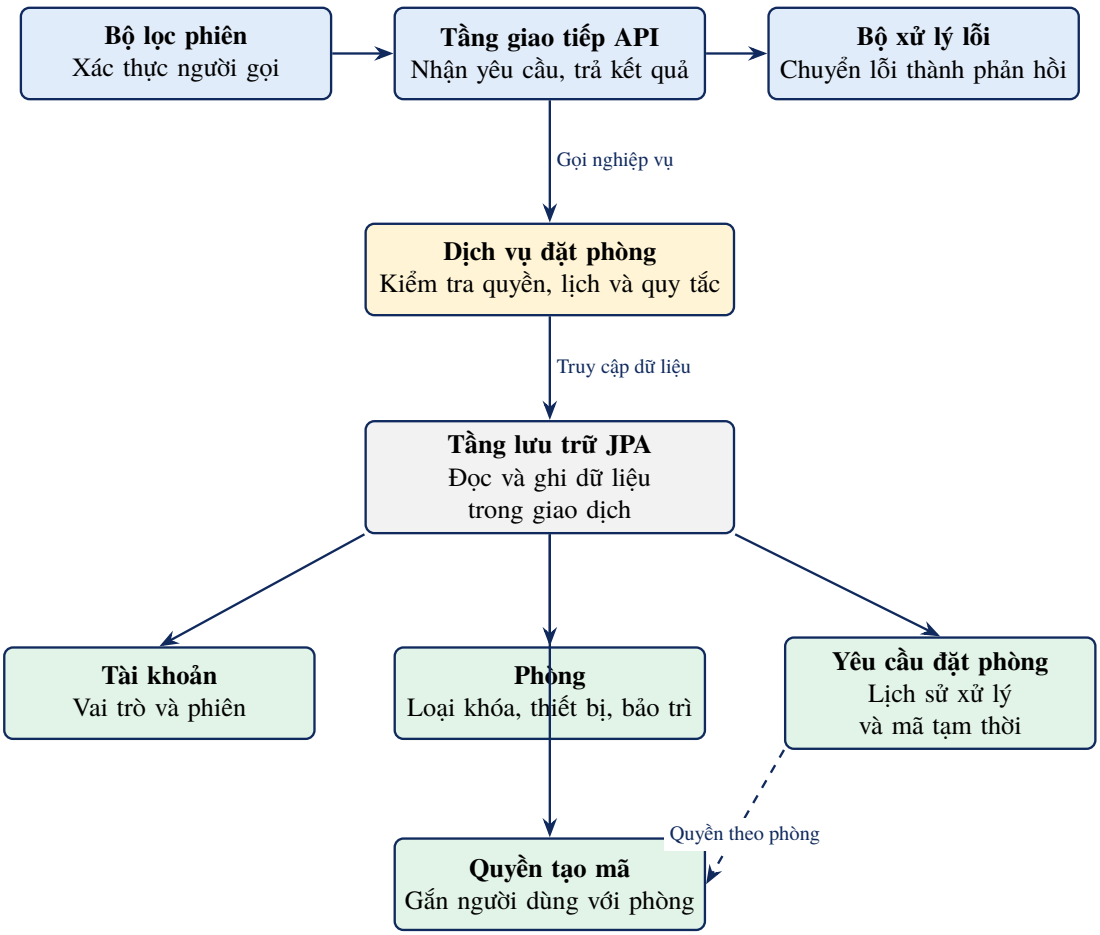


(a) Đăng nhập **(b)** Chế độ quản trị viên **(c)** Chế độ người dùng

Hình 4.3. Màn hình đăng nhập và bảng điều khiển theo vai trò

4.2.2 Thiết kế lớp

Hình 4.4 là sơ đồ rút gọn các nhóm lớp cốt lõi của dịch vụ Java. Sơ đồ trình bày trách nhiệm và chiều truy cập dữ liệu, không liệt kê thuộc tính hoặc phương thức của từng lớp.



Hình 4.4. Trách nhiệm và quan hệ giữa các nhóm lớp cốt lõi của dịch vụ Java

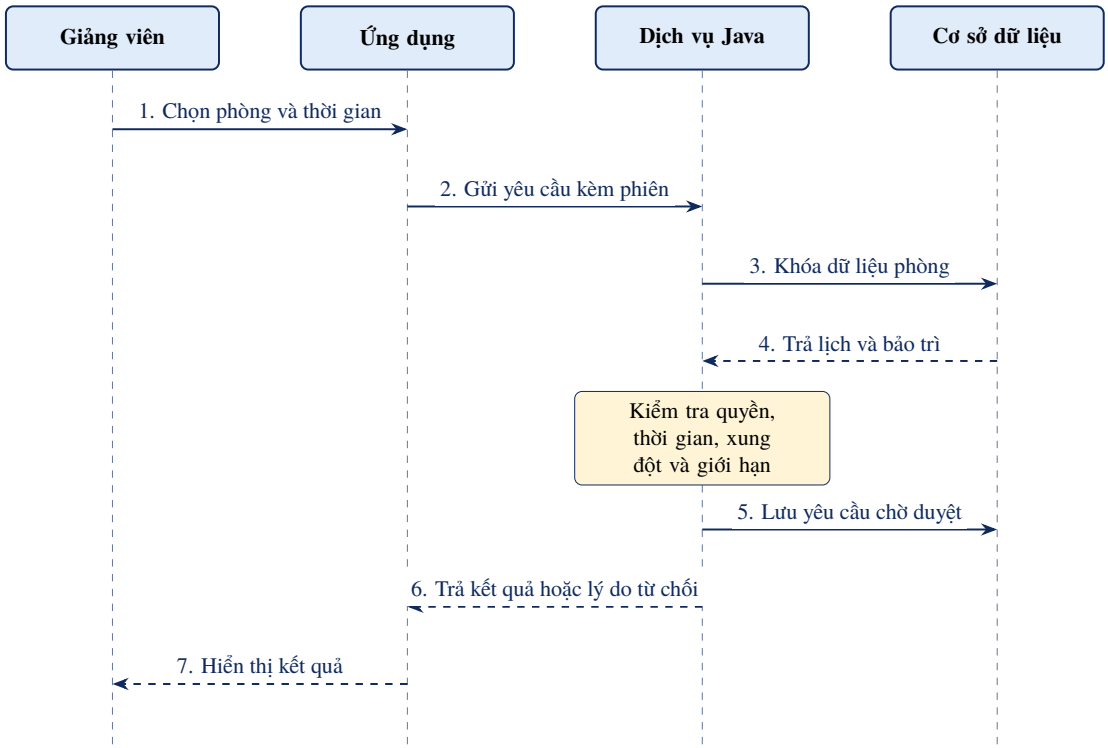
Lớp dịch vụ nghiệp vụ dùng JPA để truy cập dữ liệu và áp dụng múi giờ Việt Nam khi kiểm tra lịch. Các thao tác công khai tương ứng với ca sử dụng; các quy tắc dùng chung được gom lại để bước tạo và bước duyệt đều kiểm tra theo cùng nguyên tắc. Bảng 4.2 tóm tắt trách nhiệm chính của lớp này.

Bảng 4.2. Trách nhiệm chính của lớp dịch vụ nghiệp vụ

Nhóm trách nhiệm	Cách xử lý
Xác thực và phân quyền	Kiểm tra phiên đăng nhập, vai trò và quyền truy cập trước thao tác nghiệp vụ.
Tạo yêu cầu	Khóa dữ liệu cần thiết, kiểm tra phòng, thời gian, bảo trì và giới hạn đặt phòng rồi ghi yêu cầu chờ duyệt.
Duyệt yêu cầu	Khóa yêu cầu đích, kiểm tra lại điều kiện sử dụng, cập nhật kết quả và cấp quyền theo phòng khóa số.
Mã tạm thời	Kiểm tra chủ yêu cầu hoặc quản trị viên, định dạng mã, khoảng hiệu lực và quyền theo phòng.
Quy tắc dùng chung	Dùng phép giao khoảng thời gian và đối chiếu lịch bảo trì cho cả bước tạo lẫn bước duyệt.
Ghi dữ liệu	Sử dụng JPA trong giao dịch; tạo thông báo khi trạng thái thay đổi.

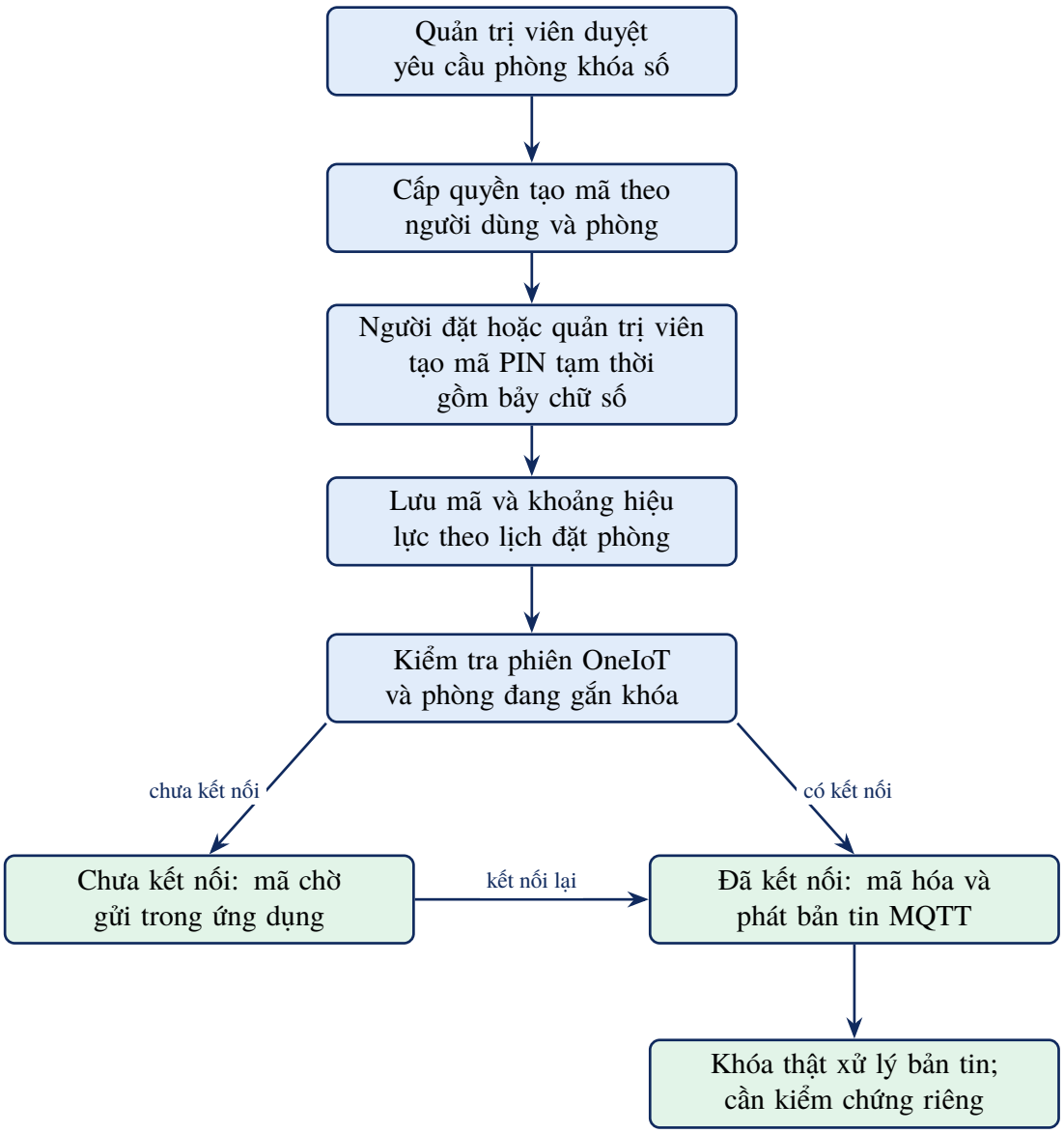
Thực thể yêu cầu đặt phòng lưu người đặt, phòng, ngày giờ sử dụng, mục đích, trạng thái và lịch sử xử lý. Thông tin mã PIN tạm thời được lưu kèm yêu cầu để xác định quyền sử dụng và khoảng hiệu lực theo đúng lịch đã duyệt. Ngày và giờ dùng định dạng thống nhất, hỗ trợ so sánh khoảng thời gian và trao đổi dữ liệu với ứng dụng.

Luồng thông điệp giữa các thành phần cho hai ca sử dụng quan trọng được thể hiện bằng biểu đồ trình tự. Hình 4.5 mô tả ca tạo yêu cầu qua máy chủ: sau bước xác thực, tầng giao tiếp chuyển dữ liệu cho tầng nghiệp vụ; tại đây hệ thống khóa dữ liệu cần thiết, kiểm tra điều kiện rồi ghi yêu cầu. Nếu vi phạm quy tắc, tầng xử lý lỗi trả phản hồi kèm lý do để ứng dụng hiển thị.



Hình 4.5. Biểu đồ trình tự ca sử dụng "Tìm và đặt phòng" qua dịch vụ Java

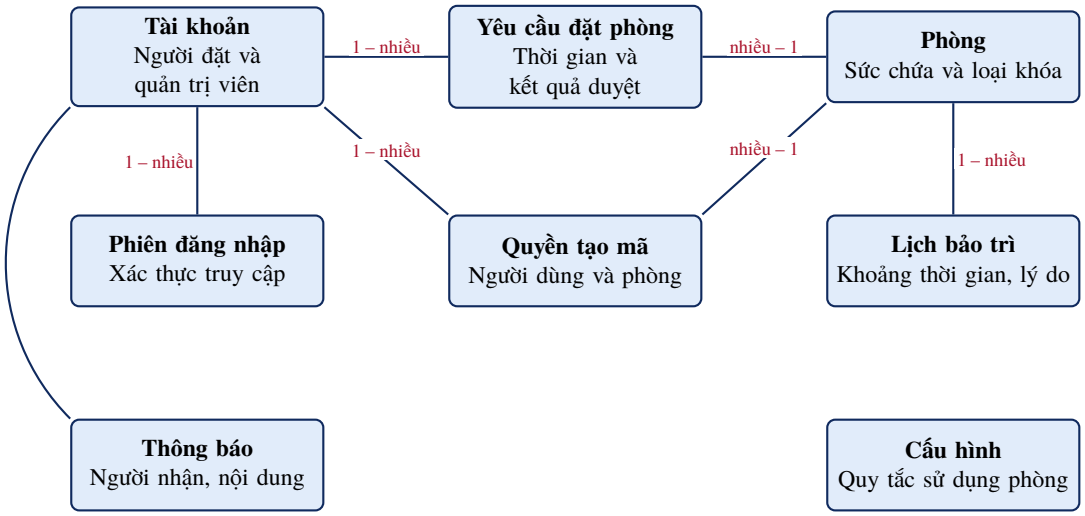
Hình 4.6 mô tả quan hệ giữa duyệt yêu cầu và cấp mã. Trong bản triển khai hiện tại, mã tạm thời được lưu trước khi gửi; nếu chưa có phiên OneIoT, lệnh được chờ tới khi kết nối. Sơ đồ trình tự là mô hình khái niệm của ca sử dụng; trình tự gửi thực tế được giải thích tại mục 5.2.



Hình 4.6. Luồng duyệt yêu cầu và tạo mã tạm thời trong bản hiện tại

4.2.3 Thiết kế cơ sở dữ liệu

Mã nguồn Java hiện khai báo 11 lớp thực thể JPA. Hình 4.7 mô tả tám nhóm dữ liệu lõi của luồng đặt phòng; dữ liệu hồ sơ, khóa đang quản lý và yêu cầu bảo trì được bổ sung sau nên không xuất hiện trong sơ đồ rút gọn. Tài khoản liên kết với phiên đăng nhập, thông báo, yêu cầu đặt phòng và quyền tạo mã; phòng liên kết với yêu cầu, lịch bảo trì và quyền theo phòng. Quyền tạo mã là quan hệ giữa người dùng và phòng, có trạng thái cấp hoặc thu hồi.



Hình 4.7. Quan hệ giữa các nhóm dữ liệu lõi của RoomHub

Danh sách thiết bị của phòng hiện được lưu gộp thành chuỗi, giúp lược đồ thử nghiệm gọn hơn nhưng chưa hỗ trợ tốt việc quản lý số lượng và tình trạng từng thiết bị. Khi triển khai các quy tắc quản lý thiết bị chi tiết, dữ liệu này cần được tách thành các bảng quan hệ riêng. Bảng 4.3 tóm tắt dữ liệu chính của một yêu cầu đặt phòng.

Bảng 4.3. Thông tin chính của yêu cầu đặt phòng

Nhóm thông tin	**Ý nghĩa**
Định danh và người đặt	Xác định yêu cầu và tài khoản chịu trách nhiệm
Phòng và thời gian	Phòng được sử dụng, ngày và khung giờ bắt đầu – kết thúc
Mục đích và trạng thái	Nội dung sử dụng; tiến trình chờ duyệt, đã duyệt, từ chối, hủy hoặc vắng mặt
Lịch sử xử lý	Thời điểm tạo, thời điểm duyệt và người xử lý
Mã PIN tạm thời	Mã, khoảng hiệu lực, vị trí mật khẩu trên khóa, định danh lệnh gửi và thời điểm thu hồi

4.3 Xây dựng ứng dụng

4.3.1 Thư viện và công cụ sử dụng

Bảng 4.4 liệt kê các ngôn ngữ, thư viện và công cụ cùng phiên bản đã dùng.

Bảng 4.4. Danh sách thư viện và công cụ sử dụng

Mục đích	Công cụ / thư viện	Địa chỉ URL
Ngôn ngữ máy chủ	Java (OpenJDK) 17	https://openjdk.org
Khung ứng dụng máy chủ	Spring Boot 3.4.4 (web, data-jpa)	https://spring.io
Cơ sở dữ liệu	H2 Database (chế độ tệp)	https://h2database.com
Quản lý build Java	Apache Maven	https://maven.apache.org
Khung ứng dụng di động	React Native 0.86.2, React 19.2.3	https://reactnative.dev
Ngôn ngữ di động	TypeScript 5.8, Kotlin 2.1.20	https://www.typescriptlang.org
Lưu trữ cục bộ	Async Storage 3.1.1	https://react-native-async-storage.github.io
Build Android	Android SDK 36, Gradle, Node.js 22	https://developer.android.com
Kiểm thử, kiểm tra mã	Jest 29, ESLint 8, Prettier 2.8	https://jestjs.io
Kết nối khóa	Module Android Kotlin, MQTT, AES-CTR	https://developer.android.com
Quản lý mã nguồn	Git, GitHub	https://github.com

4.3.2 Kết quả đạt được

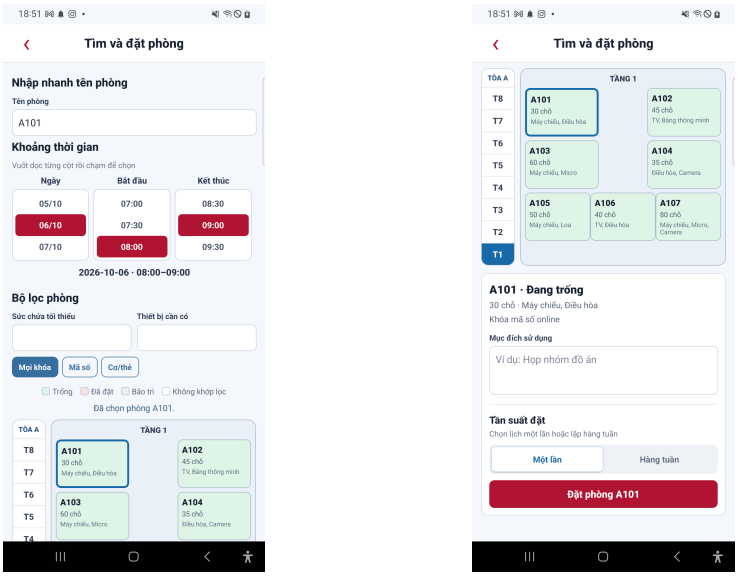
Mã nguồn dự án gồm ứng dụng Android, dịch vụ Java, công cụ mô phỏng khóa và tài liệu liên quan [20]. Ứng dụng giao tiếp OneIoT trực tiếp; mã nguồn gateway Python được giữ để thử nghiệm giao thức. APK Release có thể tạo lại từ mã nguồn. Bản trình diễn cục bộ cung cấp hai tài khoản mẫu theo vai trò quản trị viên và người dùng. Bảng 4.5 tổng hợp những thành phần có thể đối chiếu trong kho mã nguồn.

Bảng 4.5. Các thành phần và minh chứng trong mã nguồn

Thành phần	Hiện trạng có thể đối chiếu
Ứng dụng di động	Mười module nghiệp vụ; có thể đóng gói bản Android từ mã nguồn
Dịch vụ Java	Mã nguồn Spring Boot và kiểm thử tích hợp
Dữ liệu mẫu	Tám tầng, mỗi tầng bảy phòng trong cấu hình khởi tạo
Kiểm thử ứng dụng	Bảy bộ Jest, 39 ca đạt ở lần kiểm tra mã nguồn gần nhất
Giao tiếp khóa	Module Android MQTT trong ứng dụng; công cụ mô phỏng và gateway tham khảo lưu riêng

4.3.3 Minh họa các chức năng chính

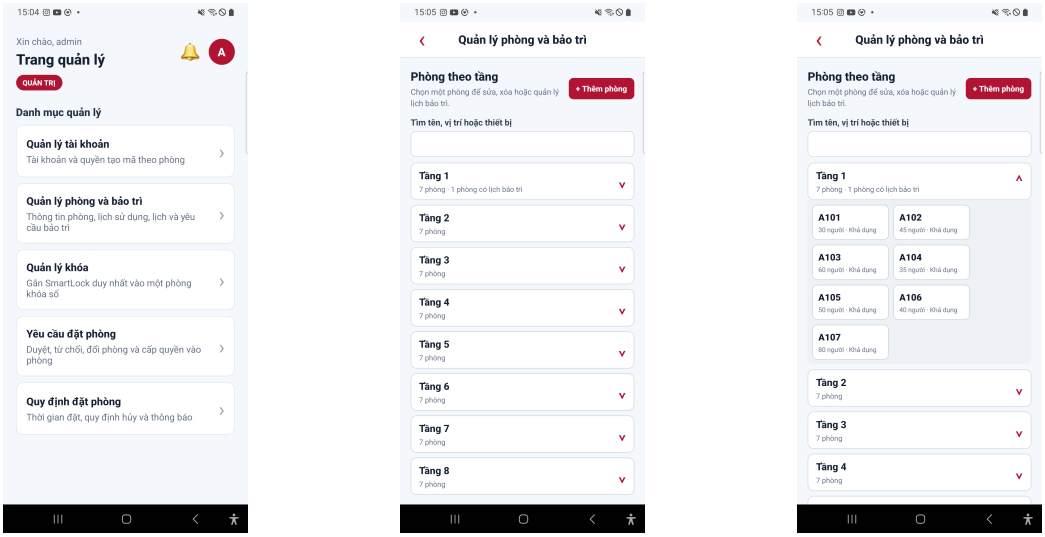
Tìm và đặt phòng trên sơ đồ tầng. Hình 4.8 minh họa thao tác chọn khung giờ, lọc phòng theo tiêu chí và đặt ngay tại phòng đã chọn. Trạng thái phòng được hiển thị theo thời gian đã chọn; người dùng không cần chuyển sang một chức năng đặt phòng riêng.



(a) Chọn khung giờ, bộ lọc và sơ đồ **(b)** Chọn phòng và gửi yêu cầu

Hình 4.8. Màn hình tìm phòng theo sơ đồ tầng và gửi yêu cầu đặt phòng

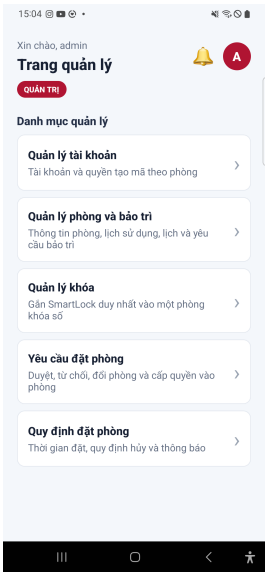
Quản lý phòng và lịch bảo trì. Chức năng quản trị gộp danh sách phòng và lịch bảo trì theo từng tầng. Quản trị viên mở tầng để xem phòng, sau đó chọn phòng để sửa, xóa hoặc xử lý bảo trì. Hình 4.9 thể hiện các cấp hiển thị từ danh sách tầng tới một phòng cụ thể.



(a) Chức năng quản trị **(b)** Nhóm theo tầng **(c)** Phòng của tầng đã chọn

Hình 4.9. Màn hình quản lý phòng và lịch bảo trì của quản trị viên

Quản lý quyền và cấu hình. Quản trị viên quản lý tài khoản và có thể thu hồi quyền tự tạo mã theo phòng. Các tham số về cửa sổ đặt trước, giới hạn yêu cầu, hạn hủy hoặc đổi phòng và khoảng đệm mã được chỉnh trong màn cấu hình. Hình 4.10 minh họa vùng chức năng quản trị và biểu mẫu cấu hình của bản hiện tại.



(a) Truy cập chức năng quản trị



(b) Cấu hình nghiệp vụ

Hình 4.10. Chức năng quản trị và cấu hình nghiệp vụ

4.4 Kiểm thử

Nhóm kiểm tra tầng dịch vụ ứng dụng bằng Jest với dữ liệu lưu trữ giả lập, bao gồm đặt phòng, quyền tạo mã, mã bảy số, gửi mã sau khi kết nối, bảo trì, đặt lặp và vắng mặt. Dịch vụ Java có kiểm thử tích hợp riêng. Bên cạnh đó, bản Release đã được kiểm tra cài đặt và khởi động trên điện thoại Android trong chế độ cục bộ. Các bảng dưới đây nêu những kịch bản tiêu biểu và kết quả mong đợi; không dùng số thứ tự bảng để suy ra số ca Jest tương ứng.

Bảng 4.6. Ca kiểm thử chức năng tạo yêu cầu đặt phòng

Mã	Đầu vào / bước	Kết quả mong đợi	Kết quả
TC-01	Người dùng đặt A101 ngày mai 08:00–09:00, mục đích “Họp nhóm đồ án”	Tạo yêu cầu chờ duyệt, quản trị viên nhận thông báo	Đạt
TC-02	Người khác đặt A101 cùng ngày 08:30–09:30	Từ chối do phòng có yêu cầu chồng giờ (BR-05)	Đạt
TC-03	Đặt A101 sau 4 ngày với cấu hình mặc định	Từ chối: “Chỉ được đặt trước từ 1 đến 3 ngày.” (BR-03)	Đạt
TC-04	Tạo bảo trì A101 08:00–09:00, đặt 08:15–08:45	Từ chối do trùng lịch bảo trì (BR-04)	Đạt
TC-05	Đổi cấu hình tối đa 5 ngày, đặt B202 sau 4 ngày	Chấp nhận yêu cầu chờ duyệt theo cấu hình mới	Đạt

Bảng 4.7. Ca kiểm thử chức năng duyệt yêu cầu và tạo mã PIN

Mã	Đầu vào / bước	Kết quả mong đợi	Kết quả
TC-06	Quản trị viên tạo mã cho yêu cầu đang chờ duyệt	Từ chối vì yêu cầu chưa được duyệt	Đạt
TC-07	Duyệt yêu cầu phòng A101 rồi tạo mã	Mã bảy chữ số; hiệu lực từ trước giờ bắt đầu 10 phút đến sau giờ kết thúc 10 phút	Đạt
TC-08	Duyệt yêu cầu phòng khóa số	Người đặt được cấp quyền tự tạo mã tại đúng phòng A101	Đạt
TC-09	Quản trị viên thu hồi quyền rồi người đặt tạo mã	Từ chối do quyền theo phòng đã bị thu hồi	Đạt
TC-10	Cấp lại quyền rồi người đặt tạo mã	Tạo thành công; ghi nhận đúng người tạo	Đạt
TC-11	Gắn khóa sang phòng khác rồi tạo mã cho A101	Từ chối vì A101 không còn gắn với khóa đang quản lý	Đạt

Bảng 4.8. Ca kiểm thử chức năng quản trị và phân quyền

Mã	Đầu vào / bước	Kết quả mong đợi	Kết quả
TC-12	Người dùng thường thử tạo tài khoản và tạo phòng	Từ chối vì không có quyền quản trị	Đạt
TC-13	Quản trị viên khóa một tài khoản, thử đăng nhập lại rồi mở khóa và khôi phục mật khẩu	Tài khoản bị khóa không đăng nhập được; sau khôi phục dùng mật khẩu mới	Đạt
TC-14	Quản trị viên tạo, sửa và xóa phòng C303	Thông tin sửa được lưu; phòng không còn trong danh sách sau khi xóa	Đạt
TC-15	Hẹn nhận khóa B202 lúc 14:00, người đặt ủy quyền người khác nhận hộ	Lưu địa điểm hẹn và thông tin định danh người nhận hộ	Đạt
TC-16	Đổi yêu cầu từ A101 sang phòng khác loại khóa, rồi sang phòng cùng loại khóa	Lần đầu bị từ chối; lần sau thành công và lưu lịch sử đổi phòng	Đạt
TC-17	Đăng nhập theo hai vai trò; thử mật khẩu sai và đăng ký tên đã tồn tại	Mở đúng giao diện khi hợp lệ; thông báo lỗi rõ ràng khi không hợp lệ	Đạt

Ở lần kiểm tra mã nguồn gần nhất, bảy bộ Jest của ứng dụng đạt 39/39 ca. Kho mã nguồn dịch vụ Java có kiểm thử tích hợp cho nhánh dữ liệu từ xa. Biên bản thiết bị lưu cùng ứng dụng ghi nhận bản Release khởi động và điều hướng theo vai trò ở chế độ cục bộ. Các kịch bản trong bảng là mẫu kiểm tra nghiệp vụ; việc nhận mã tại khóa thật và đồng bộ trên hạ tầng triển khai thực tế chưa được xác nhận bởi các kết quả này.

4.5 Triển khai

Bản Release Android có thể chạy độc lập với Metro. Cờ `ENABLE_REMOTE_SYNC` hiện đang bật và `REMOTE_API_BASE_URL` trỏ tới địa chỉ tunnel thử nghiệm. Ở chế độ

này, ứng dụng cần mạng, backend Java đang chạy và H2 phía server sẵn sàng; dữ liệu nghiệp vụ được gửi/nhận qua REST API và lưu phía server. Địa chỉ tunnel có thể thay đổi hoặc ngừng hoạt động, nên chưa được xem là endpoint production. Để demo local độc lập, chuyển cờ đồng bộ về `false` trước khi build; khi đó dữ liệu được lưu riêng trên thiết bị. Luồng khóa số kết nối OneIoT trực tiếp từ ứng dụng sau khi người dùng nhập token và kiểm tra kết nối trong phiên.

Biên bản trong kho ứng dụng xác nhận đã cài và mở bản Release trên điện thoại Android ở chế độ cục bộ, không phụ thuộc Metro. Người khác clone kho mã nguồn cần cài công cụ Android và tự build APK theo hướng dẫn tại Phụ lục B; tệp APK và cache build không được đưa vào Git.

Chương này đã trình bày kiến trúc ứng dụng và dịch vụ, thiết kế module, cơ sở dữ liệu, giao diện cập nhật, các kịch bản kiểm thử và cách triển khai theo hai chế độ. Ba giải pháp về xung đột lịch, quyền tạo mã và lựa chọn nguồn dữ liệu được phân tích tiếp ở Chương 5.

CHƯƠNG 5. CÁC GIẢI PHÁP VÀ ĐÓNG GÓP NỔI BẬT

Chương 4 đã trình bày thiết kế và kết quả xây dựng. Chương này phân tích ba giải pháp trong mã nguồn hiện tại: kiểm tra xung đột lịch khi ghi dữ liệu, cấp quyền và tạo mã tạm thời cho từng phòng, cùng cách tổ chức ứng dụng để dùng dữ liệu cục bộ hoặc dịch vụ Java. Mỗi giải pháp được trình bày theo vấn đề, cách thực hiện và phạm vi kết quả đã kiểm chứng.

5.1 Chống trùng lịch bằng phép giao khoảng thời gian và khóa bi quan

5.1.1 Vấn đề

Hai yêu cầu đặt phòng có thể cùng hợp lệ khi được kiểm tra riêng, nhưng xung đột nếu được ghi đồng thời. Cần xét cả các khoảng thời gian chạm biên: phiên 08:00–09:00 và phiên 09:00–10:00 không chiếm phòng cùng lúc. Khi nhiều người đặt cùng một phòng, việc kiểm tra rồi mới ghi còn tạo ra nguy cơ hai giao dịch cùng thấy phòng trống. Trạng thái phòng cũng có thể thay đổi trong thời gian chờ quản trị viên duyệt.

5.1.2 Giải pháp

Khoảng thời gian nửa mở. Mỗi phiên sử dụng có ngày d và khoảng giờ $[s, e)$, với $s < e$. Hai phiên trùng nhau khi

$$\mathrm{overlap}([s_1, e_1), [s_2, e_2)) \iff (d_1 = d_2) \wedge (s_1 < e_2) \wedge (s_2 < e_1). \tag{5.1}$$

Điều kiện dùng bất đẳng thức chặt nên hai phiên chỉ chạm biên được chấp nhận. Cùng phép kiểm tra được áp dụng cho yêu cầu đang hoạt động, lịch cá nhân và khoảng bảo trì [21]. Ngày và giờ được chuẩn hóa trước khi so sánh để tránh nhầm thứ tự thời gian.

Kiểm tra trong giao dịch. Dịch vụ Java dùng giao dịch và khóa bi quan ở thao tác tạo yêu cầu. Trước khi đọc lịch, dịch vụ khóa bản ghi cấu hình dùng chung và bản ghi phòng được chọn. Bản ghi phòng tồn tại độc lập với danh sách yêu cầu nên không phụ thuộc việc bảng yêu cầu đã có dữ liệu hay chưa. Ở thao tác duyệt, dịch vụ khóa yêu cầu đích rồi kiểm tra lại trạng thái và lịch phòng. Thuật toán 1 tóm tắt thứ tự xử lý.

Thuật toán 1: Tạo yêu cầu đặt phòng trong một giao dịch

Input: người đặt a, phòng r, ngày d, giờ bắt đầu s, giờ kết thúc e
Output: yêu cầu chờ duyệt hoặc thông báo lý do từ chối
kiểm tra vai trò và quyền tạo yêu cầu;
bắt đầu giao dịch; khóa bản ghi cấu hình và phòng r;
kiểm tra phòng khả dụng, định dạng thời gian và cửa sổ đặt trước;
đọc lịch bảo trì và các yêu cầu còn hiệu lực;
if *có bảo trì hoặc có khoảng giao trên cùng phòng* **then**
    từ chối;
end
if *người đặt có khoảng giờ trùng hoặc vượt giới hạn yêu cầu* **then**
    từ chối;
end
lưu yêu cầu chờ duyệt và tạo thông báo; xác nhận giao dịch;

Khóa cấu hình giúp các thao tác tạo được xếp thứ tự khi cùng dùng dịch vụ; khóa phòng xác định tài nguyên chịu xung đột. Tính đúng đắn trên một hệ quản trị cơ sở dữ liệu khác cần được kiểm tra bằng bài thử đồng thời, vì hành vi khóa phụ thuộc cấu hình giao dịch và cơ sở dữ liệu [8].

5.1.3 Kết quả đạt được

Bộ kiểm thử ứng dụng bao gồm trường hợp trùng phòng, chạm biên, bảo trì và cửa sổ đặt trước. Mã Java hiện đã có khóa trước thao tác tạo và duyệt. Đây là cơ sở kiểm soát xung đột ở dịch vụ máy chủ, nhưng báo cáo chưa coi kiểm thử cục bộ là bằng chứng cho tải đồng thời trên máy chủ triển khai thật. Bài thử nhiều yêu cầu cùng phòng cần được chạy riêng khi có hạ tầng phù hợp.

5.2 Cấp quyền tạo mã PIN theo phòng và tích hợp khóa thông minh

5.2.1 Vấn đề

Phòng khóa số cần cấp quyền cho đúng người đang có yêu cầu được duyệt. Nếu cấp quyền toàn hệ thống cho người dùng, họ có thể tạo mã tại phòng không liên quan. Ngoài ra, kết nối OneIoT có thể chưa sẵn sàng khi người dùng tạo mã; ứng dụng cần lưu trạng thái chờ gửi và tránh đánh đồng việc phát bản tin với việc khóa đã thực thi.

5.2.2 Giải pháp

Quyền theo phòng. Khi duyệt yêu cầu phòng khóa số, hệ thống cấp quyền cho cặp người dùng và phòng. Quản trị viên có thể thu hồi quyền này mà không thay đổi vai trò tài khoản. Khi tạo mã, ứng dụng kiểm tra yêu cầu đã duyệt, người tạo là chủ yêu cầu hoặc quản trị viên, quyền theo phòng còn hiệu lực đối với người dùng, và phòng khớp khóa

đang được gắn. Mỗi yêu cầu chỉ có một mã chưa bị thu hồi trong một thời điểm.

Khoảng hiệu lực. Với lịch sử dụng $[s, e)$ và khoảng đệm g trong cấu hình, mã được gắn khoảng hiệu lực

$$[t_{\text{from}}, t_{\text{until}}] = [s - g, e + g]. \tag{5.2}$$

Giá trị mặc định của g là 10 phút. Màn hình suy ra trạng thái chờ hiệu lực, đang hiệu lực, hết hạn hoặc đã thu hồi từ thời gian hiện tại và dữ liệu yêu cầu. Hệ thống không khẳng định khóa vật lý đã tự xóa mã nếu chưa có phản hồi hoặc thử nghiệm trên khóa thật.

Tạo và gửi mã. Ứng dụng sinh mã bảy chữ số, cấp một vị trí mật khẩu trên khóa rồi lưu mã cùng khoảng hiệu lực trước khi thử gửi. Nếu phiên OneIoT đang kết nối, thành phần Android mã hóa dữ liệu theo định dạng khóa và phát bản tin MQTT. Nếu chưa kết nối hoặc gửi lỗi, mã vẫn hiển thị trong ứng dụng ở trạng thái chờ; khi kết nối lại, ứng dụng thử gửi các mã còn hiệu lực, chưa thu hồi và đúng phòng gắn khóa. Thông tin xác thực OneIoT được nhập cho phiên kết nối và không nằm trong mã nguồn.

Việc mã được lưu trong ứng dụng và được đánh dấu đã phát lên broker là hai mốc khác nhau. Trạng thái đã gửi cần được phân biệt với phản hồi thực tế của khóa. Gateway Python trong kho mã nguồn là công cụ thử nghiệm giao thức; đường gửi hiện dùng trong ứng dụng là module Android.

5.2.3 Kết quả đạt được

Kiểm thử ứng dụng đã bao phủ mã bảy chữ số, quyền theo phòng, thu hồi quyền, phòng gắn khóa và trường hợp tạo mã khi chưa kết nối rồi gửi sau khi kết nối. Mã đã lưu có thể được xem trong ứng dụng; việc áp dụng trên khóa thật chưa được xác nhận chỉ bằng những kiểm thử này. Khi triển khai lâu dài cần bảo vệ tốt hơn dữ liệu mã và token, đồng thời xử lý phản hồi, gửi lại và trùng lặp bản tin.

5.3 Kiến trúc module hai chế độ dữ liệu

5.3.1 Vấn đề

Nhóm cần demo nghiệp vụ trên một điện thoại khi chưa có máy chủ, nhưng cũng cần nhánh gọi dịch vụ Java cho kịch bản nhiều thiết bị. Nếu hai chế độ dùng hai giao diện và hai bộ mã hoàn toàn riêng, việc bảo trì và đối chiếu quy tắc sẽ khó hơn. Khi dùng máy chủ, lỗi kết nối hoặc khác biệt giữa mô hình dữ liệu cục bộ và API cũng dễ làm gián đoạn luồng.

5.3.2 Giải pháp

Ứng dụng chia mã theo module, mỗi module có giao diện và tầng truy cập nghiệp vụ. Một cấu hình tập trung quyết định thao tác dùng dữ liệu cục bộ hay gọi dịch vụ Java. Giao

diện sử dụng cùng một nhóm thao tác nghiệp vụ; ở chế độ máy chủ, tầng truy cập gửi thông tin phiên, nhận dữ liệu và chuyển lỗi của máy chủ thành thông báo cho người dùng.

Bảng 5.1 đối chiếu các thao tác chính giữa ứng dụng và dịch vụ Java. Mỗi thao tác cần được kiểm tra tương ứng ở hai chế độ; báo cáo không giả định chỉ cần tên hàm giống nhau thì quy tắc đã hoàn toàn tương đương.

Bảng 5.1. Ánh xạ thao tác giữa ứng dụng và dịch vụ Java

Nhóm thao tác	**Chế độ cục bộ**	**Chế độ máy chủ**
Xác thực	Đọc tài khoản trên thiết bị	Tạo phiên và xác thực tại dịch vụ Java
Tạo và xử lý yêu cầu	Kiểm tra quy tắc trong kho dữ liệu cục bộ	Gọi REST; dịch vụ Java kiểm tra lại trong giao dịch
Quyền tạo mã	Lưu quyền theo người dùng và phòng trên thiết bị	Lưu quyền trong cơ sở dữ liệu máy chủ
Tạo mã tạm thời	Lưu mã vào dữ liệu cục bộ	Lưu mã qua API; gửi OneIoT từ ứng dụng

5.3.3 Kết quả đạt được

Mã nguồn hiện đã có backend Java, các REST API nghiệp vụ, JPA/H2 và phía app gọi API khi cờ `ENABLE_REMOTE_SYNC` bật; cấu hình hiện tại đang bật cờ này. Khi nhiều thiết bị cùng truy cập một server đang chạy và dùng chung H2 phía server, chúng có thể đọc/ghi tập dữ liệu phía server qua API. Chế độ local vẫn lưu riêng trên Async Storage của từng thiết bị và không tự đẩy hoặc hòa giải thay đổi với server khi mất mạng. Địa chỉ tunnel trong cấu hình phục vụ thử nghiệm, không bảo đảm sẵn sàng như dịch vụ production. Bộ kiểm thử Jest chạy nhánh cục bộ; dự án có kiểm thử tích hợp Java, nhưng cần kiểm thử đầu cuối nhiều thiết bị trên hạ tầng đang chạy để xác nhận triển khai thực tế.

CHƯƠNG 6. KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

6.1 Kết luận

Đề tài đã thu hẹp từ bài toán chung về quản lý tài nguyên phòng tới một sản phẩm cụ thể: RoomHub cho đặt phòng học/phòng họp trên Android. Hệ thống hiện có ứng dụng di động, backend Java/Spring Boot, REST API và cơ sở dữ liệu H2 qua JPA. Cờ trong mã nguồn đang bật chế độ remote, trong đó ứng dụng trao đổi dữ liệu nghiệp vụ với backend; chế độ local dùng Async Storage vẫn được giữ cho demo trên một thiết bị. Bản Release đã được kiểm tra chạy độc lập ở chế độ local.

Các kết quả kiểm thử phải được hiểu đúng phạm vi. Kiểm thử ứng dụng bao phủ nhiều quy tắc ở nhánh local; dự án cũng có kiểm thử tích hợp Java. Chức năng backend/API/database và luồng app–server đã được lập trình, nhưng việc vận hành phụ thuộc server, địa chỉ API và database đang sẵn sàng; cấu hình tunnel hiện tại là cấu hình thử nghiệm. Dữ liệu local không tự đồng bộ với server khi offline, và đồng bộ nhiều thiết bị cần được xác nhận đầu cuối trên hạ tầng đang chạy. Với khóa số, phát bản tin OneIoT chưa chứng minh khóa thật đã nhận hoặc áp dụng mã.

6.2 Hướng phát triển

Nhóm cần kiểm tra đầu cuối chế độ máy chủ với nhiều thiết bị, bổ sung kiểm thử đồng thời cho các trường hợp đặt cùng phòng và làm rõ cơ chế xử lý xung đột khi mạng gián đoạn. Khi triển khai lâu dài, cần rà lại lược đồ dữ liệu, bảo vệ mật khẩu tài khoản và mã khóa, quản lý phiên truy cập, và kiểm tra bảo mật của kết nối OneIoT. Một bước quan trọng khác là nhận phản hồi từ khóa thật để xác nhận trạng thái tạo mã, check-in và check-out thay vì chỉ dựa vào trạng thái gửi bản tin.

Về nghiệp vụ, có thể mở rộng quản lý nhiều khóa, theo dõi chi tiết thiết bị trong phòng, ghi nhật ký thao tác và xây dựng báo cáo sử dụng. Mỗi chức năng mới cần được kiểm chứng ở cả chế độ cục bộ và chế độ dùng máy chủ trước khi coi là hoàn thiện.

TÀI LIỆU THAM KHẢO

[1] Nhóm RoomHub, *Quy trình nghiệp vụ RoomHub, phiên bản 2.1*. Tài liệu nội bộ trong thư mục `quy_trinh_nghiep_vu`.

[2] E. Gamma, R. Helm, R. Johnson, and J. Vlissides, *Design Patterns: Elements of Reusable Object-Oriented Software*. Addison-Wesley, 1994.

[3] J. Gosling, B. Joy, G. Steele, G. Bracha, A. Buckley, D. Smith, and G. Bierman, *The Java Language Specification, Java SE 17 Edition*. Oracle America, Inc., 2021. [Online]. Available: `https://docs.oracle.com/javase/specs/jls/se17/html/index.html` (visited on 09/30/2026).

[4] J. Bloch, *Effective Java*, 3rd ed. Addison-Wesley, 2018.

[5] Spring Team, Broadcom Inc., *Spring Boot Reference Documentation, version 3.4*. [Online]. Available: `https://docs.spring.io/spring-boot/3.4/index.html` (visited on 09/30/2026).

[6] Eclipse Foundation, *Jakarta Persistence 3.1 Specification*. [Online]. Available: `https://jakarta.ee/specifications/persistence/3.1/` (visited on 09/30/2026).

[7] M. Fowler, *Patterns of Enterprise Application Architecture*. Addison-Wesley, 2002.

[8] P. A. Bernstein and E. Newcomer, *Principles of Transaction Processing*, 2nd ed. Morgan Kaufmann, 2009.

[9] H2 Group, *H2 Database Engine – Documentation*. [Online]. Available: `https://www.h2database.com/html/main.html` (visited on 09/30/2026).

[10] R. T. Fielding, "Architectural styles and the design of network-based software architectures," Ph.D. dissertation, University of California, Irvine, USA, 2000.

[11] Meta Platforms, Inc., *React Native Documentation*. [Online]. Available: `https://reactnative.dev/docs/getting-started` (visited on 09/30/2026).

[12] Microsoft Corporation, *The TypeScript Handbook*. [Online]. Available: `https://www.typescriptlang.org/docs/handbook/intro.html` (visited on 09/30/2026).

[13] React Native Community, *Async Storage Documentation*. [Online]. Available: `https://react-native-async-storage.github.io/async-storage/` (visited on 09/30/2026).

[14] OASIS, *MQTT Version 5.0, OASIS Standard*, 2019. [Online]. Available: `https://docs.oasis-open.org/mqtt/mqtt/v5.0/mqtt-v5.0.html` (visited on 09/30/2026).

[15] oneM2M Partners, *TS-0010 MQTT Protocol Binding*. [Online]. Available: `https://www.onem2m.org/technical/published-specifications` (visited on 09/30/2026).

[16] M. Dworkin, "Recommendation for block cipher modes of operation: Methods and techniques," National Institute of Standards and Technology, NIST Special Publication 800-38A, 2001.

[17] Object Management Group, *Unified Modeling Language (UML) Specification, Version 2.5.1*, 2017. [Online]. Available: `https://www.omg.org/spec/UML/2.5.1` (visited on 09/30/2026).

[18] OpenJS Foundation, *Jest Documentation*. [Online]. Available: `https://jestjs.io/docs/getting-started` (visited on 09/30/2026).

[19] R. C. Martin, *Clean Architecture: A Craftsman's Guide to Software Structure and Design*. Prentice Hall, 2017.

[20] Nhóm RoomHub, *Mã nguồn RoomHub – classroom-booking*. [Online]. Available: `https://github.com/xwomen1/classroom-booking` (visited on 09/30/2026).

[21] J. F. Allen, "Maintaining knowledge about temporal intervals," *Communications of the ACM*, vol. 26, no. 11, pp. 832–843, 1983.

Phụ lục A. ĐẶC TẢ USE CASE BỔ SUNG

Phụ lục này đặc tả các use case không được trình bày trong mục 2.3. Mã quy tắc BR tham chiếu tới Bảng 2.3.

A.1 Đặc tả use case "Hủy đặt phòng"

Bảng A.1. Đặc tả use case UC06 – Hủy đặt phòng

Tác nhân	Người đặt phòng
Tiền điều kiện	Yêu cầu đang chờ hoặc đã duyệt, chưa đến giờ bắt đầu.
Luồng chính	1. Người đặt mở danh sách yêu cầu của mình, chọn yêu cầu và bấm hủy. 2. Hệ thống kiểm tra quyền sở hữu và tình trạng của yêu cầu. 3. Với yêu cầu đã duyệt, hệ thống kiểm tra còn ít nhất 30 phút (BR-08). 4. Hệ thống chuyển sang trạng thái đã hủy và ghi thời điểm thu hồi mã PIN nếu có. 5. Hệ thống thông báo kết quả cho quản trị viên.
Luồng phát sinh	3a. Quá hạn hủy: báo "Yêu cầu đã duyệt chỉ được hủy trước ít nhất 30 phút". 2a. Phiên đã bắt đầu: báo không thể hủy.
Hậu điều kiện	Yêu cầu đã hủy; phòng trở lại trạng thái trống trong khung giờ đó và mã PIN bị thu hồi.

A.2 Đặc tả use case "Hẹn lịch nhận khóa và ủy quyền nhận hộ"

Bảng A.2. Đặc tả use case UC07 – Hẹn lịch nhận khóa và ủy quyền nhận hộ

Tác nhân	Quản trị viên, người đặt phòng
Tiền điều kiện	Yêu cầu đã duyệt tại phòng dùng khóa cơ hoặc thẻ, chưa đến giờ sử dụng.
Luồng chính	1. Người đặt đề xuất thời điểm nhận khóa trước giờ sử dụng. 2. Quản trị viên chấp nhận hoặc đề xuất thời điểm, địa điểm khác; hai bên tiếp tục phản hồi tới khi thống nhất. 3. Nếu không tự đến nhận, người đặt ghi họ tên và mã định danh của người nhận hộ. 4. Hệ thống lưu lịch hẹn đã thống nhất và thông báo cho các bên; người nhận hộ xuất trình giấy tờ khi nhận.
Luồng phát sinh	2a. Phòng dùng khóa số: báo "Phòng này không dùng khóa cơ/thẻ". 3a. Thiếu họ tên hoặc mã: báo lỗi.
Hậu điều kiện	Lịch hẹn nhận khóa được lưu cùng thông tin người nhận hộ, nếu có.

A.3 Đặc tả use case "Quản lý lịch sử dụng và bảo trì"

Bảng A.3. Đặc tả use case UC08 – Quản lý lịch sử dụng và bảo trì

Tác nhân	Quản trị viên
Tiền điều kiện	Quản trị viên đã đăng nhập.
Luồng chính	1. Quản trị viên mở lịch phòng và bảo trì để xem các phiên sử dụng đã được duyệt. 2. Quản trị viên chọn tầng, phòng, ngày giờ và nhập lý do bảo trì. 3. Hệ thống kiểm tra phòng tồn tại, khoảng giờ hợp lệ, có lý do và không trùng bảo trì hoặc phiên sử dụng đã duyệt. 4. Hệ thống lưu lịch bảo trì; phòng hiển thị màu bảo trì trên sơ đồ trong khung giờ đó.
Luồng phát sinh	3a. Trùng phiên đã duyệt: yêu cầu đổi phòng trước khi lập lịch bảo trì. 4a. Quản trị viên hủy lịch bảo trì: ghi thời điểm hủy và cập nhật trạng thái phòng.
Hậu điều kiện	Các yêu cầu mới trùng khung giờ bị chặn theo BR-04.

A.4 Đặc tả use case "Cấu hình nghiệp vụ"

Bảng A.4. Đặc tả use case UC09 – Cấu hình nghiệp vụ

Tác nhân	Quản trị viên
Tiền điều kiện	Quản trị viên đã đăng nhập.
Luồng chính	1. Quản trị viên mở phần quy định đặt phòng; hệ thống hiển thị giá trị hiện tại. 2. Quản trị viên sửa cửa sổ đặt trước, số yêu cầu tối đa của mỗi người, hạn hủy, hạn đổi phòng, khoảng đệm mã số và tùy chọn thông báo. 3. Hệ thống kiểm tra các giá trị là số nguyên không âm, thời hạn tối thiểu không lớn hơn tối đa và giới hạn yêu cầu ít nhất bằng một. 4. Hệ thống lưu cấu hình; các module đọc giá trị mới ở lần kiểm tra tiếp theo.
Luồng phát sinh	3a. Giá trị không hợp lệ: hiển thị thông điệp tương ứng, không lưu.
Hậu điều kiện	Quy tắc BR-03, BR-06, BR-08, BR-09, BR-10 dùng tham số mới.

A.5 Đặc tả use case "Quản lý SmartLock"

Bảng A.5. Đặc tả use case UC10 – Quản lý SmartLock

Tác nhân	Quản trị viên; nền tảng OneIoT
Tiền điều kiện	Quản trị viên đã đăng nhập; ứng dụng có thông tin khóa số đang quản lý.
Luồng chính	1. Quản trị viên mở phần quản lý khóa, chọn phòng khóa số để gắn với khóa đang quản lý. 2. Hệ thống kiểm tra loại khóa của phòng và lưu liên kết phòng – khóa. 3. Khi cần kết nối, người dùng nhập token OneIoT cho phiên hiện tại và bấm kiểm tra kết nối. 4. Ứng dụng hiển thị trạng thái kết nối; khi có phiên kết nối, các mã đang chờ và còn hiệu lực có thể được gửi.
Luồng phát sinh	1a. Chọn phòng khóa cơ hoặc thẻ: báo không phù hợp. 3a. Thông tin xác thực không hợp lệ hoặc mạng lỗi: chưa kết nối, mã vẫn chờ gửi. Quản trị viên có thể tháo liên kết khóa khỏi phòng.
Hậu điều kiện	Chỉ yêu cầu tại phòng đang gắn khóa mới được gửi lệnh tạo mật khẩu.

A.6 Đặc tả use case "Quản lý tài khoản và quyền theo phòng"

Bảng A.6. Đặc tả use case UC11 – Quản lý tài khoản và quyền theo phòng

Tác nhân	Quản trị viên
Tiền điều kiện	Quản trị viên đã đăng nhập.
Luồng chính	1. Quản trị viên nhập tên đăng nhập, mật khẩu ban đầu và mã khôi phục để thêm tài khoản. 2. Hệ thống kiểm tra tên 3–30 ký tự thường, số, dấu chấm, gạch ngang; mật khẩu và mã khôi phục ít nhất 4 ký tự; tên chưa tồn tại. 3. Quản trị viên tìm tài khoản, khóa, mở, đặt lại mật khẩu hoặc xóa. 4. Quản trị viên xem quyền tự tạo mã của người dùng và có thể thu hồi theo từng phòng.
Luồng phát sinh	3a. Tự khóa hoặc tự xóa tài khoản đang đăng nhập: bị chặn. 3b. Tài khoản còn yêu cầu đang hoạt động: không cho xóa.
Hậu điều kiện	Tài khoản bị khóa không đăng nhập được; quyền bị thu hồi chặn tạo mã theo BR-11.

Phụ lục B. DANH SÁCH API, CÀI ĐẶT VÀ MINH CHỨNG KIỂM THỬ

B.1 Danh sách REST API của dịch vụ Java

Dịch vụ Java cung cấp API cho xác thực, phòng, yêu cầu đặt phòng, bảo trì, cấu hình, quyền tạo mã và thông báo. Các thao tác cần phiên đăng nhập được kiểm tra bằng token Bearer; đăng ký và đăng nhập có quy tắc xác thực riêng. Bảng B.1 nêu các nhóm điểm cuối tiêu biểu, không phải danh sách đầy đủ của mã nguồn hiện tại.

Bảng B.1. Các nhóm REST API tiêu biểu của dịch vụ Java

Phương thức	**Đường dẫn**	**Chức năng**
POST	`/api/auth/login`	Đăng nhập, tạo phiên
GET	`/api/rooms`	Đọc danh sách phòng
GET	`/api/configuration`	Đọc quy tắc nghiệp vụ
GET	`/api/maintenance`	Đọc lịch bảo trì
GET, POST	`/api/bookings`	Đọc và tạo yêu cầu
GET	`/api/room-occupancy`	Đọc khung giờ phòng đang chờ duyệt hoặc đã duyệt
POST	`/api/bookings/{id}/review`	Duyệt hoặc từ chối
POST	`/api/bookings/{id}/temporary-pin`	Lưu mã tạm thời và thông tin giao lệnh
GET	`/api/pin-permissions`	Xem quyền theo phòng
GET	`/api/notifications`	Đọc thông báo

Ứng dụng hiện giao tiếp OneIoT trực tiếp qua module Android. Các điểm cuối HTTP của gateway Python là công cụ thử nghiệm độc lập, không nằm trong đường truyền chính của ứng dụng.

B.2 Hướng dẫn cài đặt và chạy thử

Build bản cục bộ. Clone kho mã nguồn, vào thư mục ứng dụng và đặt `ENABLE_REMOTE_SYNC` thành `false` trong tệp cấu hình trước khi build nếu muốn demo không cần máy chủ. Mã nguồn tại thời điểm rà soát đang bật chế độ máy chủ; ở cấu hình này APK cần endpoint API và server đang hoạt động.

```
git clone https://github.com/xwomen1/classroom-booking.git
cd classroom-booking/code/booking_classroom
npm ci
npm run build:android:release
```

```
adb install -r android/app/build/outputs/apk/release/app-
release.apk
```

Tài khoản mẫu của dữ liệu cục bộ là `admin/1` và `user/2`. APK Release sau build chạy độc lập với Metro; kho Git chỉ cần mã nguồn và tài liệu build, không cần lưu tệp APK hay thư mục cache.

Chạy bản phát triển. Trong thư mục ứng dụng, chạy `npm start`; ở cửa sổ khác chạy `adb reverse tcp:8081 tcp:8081` rồi `npm run android`. Kiểm tra mã nguồn bằng `npm test -- --runInBand`, `npm run lint` và `npx tsc --noEmit`.

Chạy dịch vụ Java. Tại `code/booking-server`, chạy `mvn test` và `mvn spring-boot:run`. Server dùng Spring Data JPA và H2 ở chế độ tệp theo cấu hình mặc định; có thể đặt URL, tài khoản và mật khẩu qua biến `BOOKING_DB_URL`, `BOOKING_DB_USERNAME` và `BOOKING_DB_PASSWORD`. Để app gọi máy chủ, bật cờ đồng bộ và đặt địa chỉ API có thể truy cập từ điện thoại. Cần kiểm tra kết nối, phiên đăng nhập và dữ liệu đọc/ghi; cờ bật không bảo đảm server hoặc tunnel luôn sẵn sàng.

Kết nối khóa số. Gắn khóa đang quản lý với một phòng khóa số trong ứng dụng. Khi cần dùng OneIoT, nhập token cho phiên hiện tại và bấm kiểm tra kết nối. Token không được ghi trong mã nguồn hoặc đưa lên Git. Khi ứng dụng không còn phiên kết nối, mã được tạo vẫn có thể ở trạng thái chờ gửi; chỉ các mã còn hiệu lực của phòng đang gắn khóa mới được thử gửi khi kết nối lại.

B.3 Biên bản kiểm tra trên thiết bị

Bảng B.2 tóm tắt phạm vi minh chứng trong biên bản thiết bị của ứng dụng. Kết quả được ghi cho chế độ cục bộ; kiểm thử khóa vật lý và hệ thống nhiều thiết bị là hạng mục riêng.

Bảng B.2. Phạm vi kiểm tra bản Release trên điện thoại Android

Hạng mục	**Ghi nhận**
Cài và khởi động bản Release không cần Metro	Đã kiểm tra
Đăng nhập tài khoản mẫu, điều hướng theo vai trò	Đã kiểm tra
Danh sách phòng theo tầng và chức năng đặt phòng cục bộ	Đã kiểm tra
Kiểm thử tự động ứng dụng ở lần rà soát gần nhất	39/39 ca đạt
Kiểm thử khóa thật áp dụng mã và đồng bộ nhiều điện thoại	Chưa có xác nhận đầu cuối